

difficult to implement than the ignore-the-new-item policy, because it requires that we modify the data structure.

To implement generalized queues with no duplicate items, we assume that we have an abstract operation for testing item equality, as discussed in Section 4.1. Given such an operation, we still need to be able to determine whether a new item to be inserted is already in the data structure. This general case amounts to implementing the symbol table ADT, so we shall consider it in the context of the implementations given in Chapters 12 through 15.

There is an important special case for which we have a straightforward solution, which is illustrated for the pushdown stack ADT in Program 4.12. This implementation assumes that the items are integers in the range 0 to $M - 1$. Then, it uses a second array, indexed by the item itself, to determine whether that item is in the stack. When we insert item i , we set the i th entry in the second array to 1; when we delete item i , we set the i th entry in the array to 0. Otherwise, we use the same code as before to insert and delete items, with one additional test: Before inserting an item, we can test to see whether it is already in the stack. If it is, we ignore the *push*. This solution does not depend on whether we use an array or linked-list (or some other) representation for the stack. Implementing an ignore-the-old-item policy involves more work (see Exercise 4.51).

In summary, one way to implement a stack with no duplicates using an ignore-the-new-item policy is to maintain *two* data structures: the first contains the items in the stack, as before, to keep track of the order in which the items in the stack were inserted; the second is an array that allows us to keep track of which items are in the stack, by using the item as an index. Using an array in this way is a special case of a symbol-table implementation, which is discussed in Section 12.2. We can apply the same technique to any generalized queue ADT, when we know the items to be integers in the range 0 to $M - 1$.

This special case arises frequently. The most important example is when the items in the data structure are themselves array indices, so we refer to such items as *index items*. Typically, we have a set of M objects, kept in yet another array, that we need to pass through a generalized queue structure as a part of a more complex algorithm. Objects are put on the queue by index and processed when they are

```

F      F
I      F I
R      F I R
S      F I R S
* F    I R S
T      I R S T
* I    R S T
I      R S T I
N      R S T I N
* R    S T I N
* S    T I N
* T    I N
F      I N F
I      N F I
* N    F I
R      F I R
S      F I R S
* F    I R S
* I    R S
* R    S
T      S T
* S    T
O      T O
U      T O U
T      O U T
* O    U T
* U    T
* T

```

Figure 4.11
FIFO queue with no duplicates, forget-the-old-item policy

This sequence shows the result of the same operations as in Figure 4.10, but using the (more difficult to implement) policy by which we always add a new item at the end of the queue. If there is a duplicate, we remove it.

Program 4.12 Stack with index items and no duplicates

This pushdown-stack implementation assumes that all items are integers between 0 and `maxN-1`, so that it can maintain an array `t` that has a nonzero value corresponding to each item in the stack. The array enables `STACKpush` to test quickly whether its argument is already on the stack, and to take no action if the test succeeds. We use only one bit per entry in `t`, so we could save space by using characters or bits instead of integers, if desired (see Exercise 12.12).

```
#include <stdlib.h>
static int *s, *t;
static int N;
void STACKinit(int maxN)
{ int i;
  s = malloc(maxN*sizeof(int));
  t = malloc(maxN*sizeof(int));
  for (i = 0; i < maxN; i++) t[i] = 0;
  N = 0;
}
int STACKempty()
{ return !N; }
void STACKpush(int item)
{
  if (t[item] == 1) return;
  s[N++] = item; t[item] = 1;
}
int STACKpop()
{ N--; t[s[N]] = 0; return s[N]; }
```

removed, and each object is to be processed precisely once. Using array indices in a queue with no duplicates accomplishes this goal directly.

Each of these choices (disallow duplicates, or do not; and use the new item, or do not) leads to a new ADT. The differences may seem minor, but they obviously affect the dynamic behavior of the ADT as seen by client programs, and affect our choice of algorithm and data structure to implement the various operations, so we have no alternative but to treat all the ADTs as different. Furthermore, we have other options to consider: For example, we might wish to modify

the interface to inform the client program when it attempts to insert a duplicate item, or to give the client the option whether to ignore the new item or to forget the old one.

When we informally use a term such as *pushdown stack*, *FIFO queue*, *deque*, *priority queue*, or *symbol table*, we are potentially referring to a *family* of ADTs, each with different sets of defined operations and different sets of conventions about the meanings of the operations, each requiring different and, in some cases, more sophisticated implementations to be able to support those operations efficiently.

Exercises

- ▷ 4.47 Draw a figure corresponding to Figure 4.9 for the stack ADT that disallows duplicates using a forget-the-old-item policy.
- 4.48 Modify the standard array-based stack implementation in Section 4.4 (Program 4.4) to disallow duplicates with an ignore-the-new-item policy. Use a brute-force approach that involves scanning through the whole stack.
- 4.49 Modify the standard array-based stack implementation in Section 4.4 (Program 4.4) to disallow duplicates with a forget-the-old-item policy. Use a brute-force approach that involves scanning through, and possibly rearranging, the whole stack.
- 4.50 Do Exercises 4.48 and 4.49 for the linked-list-based stack implementation in Section 4.4 (Program 4.5).
- 4.51 Develop a pushdown-stack implementation that disallows duplicates, using a forget-the-old-item policy for integer items between 0 and $M - 1$, and that uses constant time for both *push* and *pop*. *Hint*: Use a doubly linked list representation for the stack and keep pointers to nodes, rather than 0–1 values, in an item-indexed array.
- 4.52 Do Exercises 4.48 and 4.49 for the FIFO queue ADT.
- 4.53 Do Exercise 4.50 for the FIFO queue ADT.
- 4.54 Do Exercise 4.51 for the FIFO queue ADT.
- 4.55 Do Exercises 4.48 and 4.49 for the randomized-queue ADT.
- 4.56 Write a client program for your ADT from Exercise 4.55, which exercises a randomized queue with no duplicates.

4.8 First-Class ADTs

Our interfaces and implementations of stack and FIFO queue ADTs in Sections 4.2 through 4.7 provide clients with the capability to use

a *single* instance of a particular generalized stack or queue, and to achieve the important objective of hiding from the client the particular data structure used in the implementation. Such ADTs are widely useful, and will serve as the basis for many of the implementations that we consider in this book.

These objects are disarmingly simple when considered as ADTs themselves, however, because there is only one object in a given program. The situation is analogous to having a program, for example, that manipulates only one integer. We could perhaps increment, decrement, and test the value of the integer, but could not declare variables or use it as an argument or return value in a function, or even multiply it by another integer. In this section, we consider how to construct ADTs that we can manipulate in the same way that we manipulate built-in types in client programs, while still achieving the objective of hiding the implementation from the client.

Definition 4.4 *A first-class data type is one for which we can have potentially many different instances, and which we can assign to variables which we can declare to hold the instances.*

For example, we could use first-class data types as arguments and return values to functions.

The method that we will use to implement first-class data types applies to any data type: in particular, it applies to generalized queues, so it provides us with the capability to write programs that manipulate stacks and FIFO queues in much the same way that we manipulate other types of data in C. This capability is important in the study of algorithms because it provides us with a natural way to express high-level operations involving such ADTs. For example, we can speak of operations to *join* two queues—to combine them into one. We shall consider algorithms that implement such operations for the priority queue ADT (Chapter 9) and for the symbol table ADT (Chapter 12).

Some modern languages provide specific mechanisms for building first-class ADTs, but the idea transcends specific mechanisms. Being able to manipulate instances of ADTs in much the same way that we manipulate built-in data types such as `int` or `float` is an important goal in the design of many high-level programming languages, because it allows any applications program to be written such that the program manipulates the objects of central concern to the application; it allows

many programmers to work simultaneously on large systems, all using a precisely defined set of abstract operations; and it provides for those abstract operations to be implemented in many different ways without any changes to the applications code—for example for new machines and programming environments. Some languages even allow *operator overloading*, which allows us to use basic symbols such as $+$ or $*$ to define operators. C does not provide specific support for building first-class data types, but it does provide primitive operations that we can use to achieve that goal. There are a number of ways to proceed in C. To keep our focus on algorithms and data structures, as opposed to programming-language design issues, we do not consider all the alternatives; rather, we describe and adopt just one convention that we can use throughout the book.

To illustrate the basic approach, we begin by considering, as an example, a first-class data type and then a first-class ADT for the *complex-number* abstraction. Our goal is to be able to write programs like Program 4.13, which performs algebraic operations on complex numbers using operations defined in the ADT. We implement the *add* and *multiply* operations as standard C functions, since C does not support operator overloading.

Program 4.13 uses few properties of complex numbers; we now digress to consider these properties briefly. In one sense, we are not digressing at all, because it is interesting to contemplate the relationship between complex numbers themselves as a mathematical abstraction and this abstract representation of them in a computer program.

The number $i = \sqrt{-1}$ is an *imaginary* number. Although $\sqrt{-1}$ is meaningless as a real number, we name it i , and perform algebraic manipulations with i , replacing i^2 with -1 whenever it appears. A *complex number* consists of two parts, real and imaginary—complex numbers can be written in the form $a + bi$, where a and b are reals. To multiply complex numbers, we apply the usual algebraic rules, replacing i^2 with -1 whenever it appears. For example,

$$(a + bi)(c + di) = ac + bci + adi + bdi^2 = (ac - bd) + (ad + bc)i.$$

The real or imaginary parts might cancel out (have the value 0) when we perform a complex multiplication. For example,

$$(1 - i)(1 - i) = 1 - i - i + i^2 = -2i,$$

$$(1 + i)^4 = 4i^2 = -4,$$

Program 4.13 Complex numbers driver (roots of unity)

This client program performs a computation on complex numbers using an ADT that allows it to compute directly with the abstraction of interest by declaring variables of type `Complex` and using them as arguments and return values of functions. This program checks the ADT implementation by computing the powers of the roots of unity. It prints the table shown in Figure 4.12.

```
#include <stdio.h>
#include <math.h>
#include "COMPLEX.h"
#define PI 3.141592625
main(int argc, char *argv[])
{ int i, j, N = atoi(argv[1]);
  Complex t, x;
  printf("%dth complex roots of unity\n", N);
  for (i = 0; i < N; i++)
  { float r = 2.0*PI*i/N;
    t = COMPLEXinit(cos(r), sin(r));
    printf("%2d %6.3f %6.3f ", i, Re(t), Im(t));
    for (x = t, j = 0; j < N-1; j++)
      x = COMPLEXmult(t, x);
    printf("%6.3f %6.3f\n", Re(x), Im(x));
  }
}
```

$$(1 + i)^8 = 16.$$

Scaling the preceding equation by dividing through by $16 = (\sqrt{2})^8$, we find that

$$\left(\frac{1}{\sqrt{2}} + \frac{i}{\sqrt{2}} \right)^8 = 1.$$

In general, there are many complex numbers that evaluate to 1 when raised to a power. These are the *complex roots of unity*. Indeed, for each N , there are exactly N complex numbers z with $z^N = 1$. The numbers

$$\cos\left(\frac{2\pi k}{N}\right) + i \sin\left(\frac{2\pi k}{N}\right),$$

Program 4.14 First-class data type for complex numbers

This interface for complex numbers includes a `typedef` that allows implementations to declare variables of type `Complex` and to use these variables as function arguments and return values. However, the data type is not abstract, because this representation is not hidden from clients.

```
typedef struct { float Re; float Im; } Complex;
Complex COMPLEXinit(float, float);
float Re(Complex);
float Im(Complex);
Complex COMPLEXmult(Complex, Complex);
```

for $k = 0, 1, \dots, N - 1$ are easily shown to have this property (see Exercise 4.63). For example, taking $k = 1$ and $N = 8$ in this formula gives the particular eighth root of unity that we just discovered.

Program 4.13 is an example of a client program for the complex-numbers ADT that raises each of the N th roots of unity to the N th power, using the multiplication operation defined in the ADT. The output that it produces is shown in Figure 4.12: We expect that each number raised to the N th power gives the same result: 1, or $1 + 0i$.

This client program differs from the client programs that we have considered to this point in one major respect: it declares variables of type `Complex` and assigns values to such variables, including using them as arguments and return values in functions. Accordingly, we need to define the type `Complex` in the interface.

Program 4.14 is an interface for complex numbers that we might consider using. It defines the type `Complex` as a `struct` comprising two floats (for the real and imaginary part of the complex number), and declares four functions for processing complex numbers: initialize, extract real and imaginary parts, and multiply. Program 4.15 gives implementations of these functions, which are straightforward. Together, these two functions provide an effective implementation of a complex-number ADT that we can use successfully in client programs such as Program 4.13.

The interface in Program 4.14 specifies one particular representation for complex numbers—a structure containing two integers (the real and imaginary parts). By including this representation within the

0	1.000	0.000	1.000	0.000
1	0.707	0.707	1.000	0.000
2	-0.000	1.000	1.000	0.000
3	-0.707	0.707	1.000	0.000
4	-1.000	-0.000	1.000	0.000
5	-0.707	-0.707	1.000	-0.000
6	0.000	-1.000	1.000	0.000
7	0.707	-0.707	1.000	-0.000

Figure 4.12
Complex roots of unity

This table gives the output that is produced by Program 4.13 when invoked with `a.out 8`. The eight complex roots of unity are ± 1 , $\pm i$, and

$$\pm \frac{\sqrt{2}}{2} \pm \frac{\sqrt{2}}{2}i$$

(left two columns). Each of these eight numbers gives the result $1 + 0i$ when raised to the eighth power (right two columns).

Program 4.15 Complex-numbers data-type implementation

These function implementations for the complex numbers data type are straightforward. However, we would prefer not to separate them from the definition of the `Complex` type, which is defined in the interface for the convenience of the client.

```
#include "COMPLEX.h"
Complex COMPLEXinit(float Re, float Im)
{ Complex t; t.Re = Re; t.Im = Im; return t; }
float Re(Complex z)
{ return z.Re; }
float Im(Complex z)
{ return z.Im; }
Complex COMPLEXmult(Complex a, Complex b)
{ Complex t;
  t.Re = a.Re*b.Re - a.Im*b.Im;
  t.Im = a.Re*b.Im + a.Im*b.Re;
  return t;
}
```

interface, however, we are making it available for use by client programs. Programmers often organize interfaces in this way. Essentially, doing so amounts to publishing a standard representation for a new data type that might be used by many client programs. In this example, client programs could refer directly to `t.Re` and `t.Im` for any variable `t` of type `Complex`. The advantage of allowing such access is that we thus ensure that clients that need to directly implement their own manipulations that may not be present in the type's suite of operations at least agree on the standard representation. The disadvantage of allowing clients direct access to the data is that we cannot change the representation without changing all the clients. In short, Program 4.14 is not an *abstract* data type, because the representation is not hidden by the interface.

Even for this simple example, the difficulty of changing representations is significant because there is another standard representation that we might wish to consider using: polar coordinates (see Exercise 4.62). For an application with more complicated data structures, the ability to change representations is a requirement. For example,

Program 4.16 First-class ADT for complex numbers

This interface provides clients with handles to complex number objects, but does not give any information about the representation—it is a struct that is not specified, except for its tag name.

```
typedef struct complex *Complex;
Complex COMPLEXinit(float, float);
float Re(Complex);
float Im(Complex);
Complex COMPLEXmult(Complex, Complex);
```

our company that needs to process mailing lists needs to use the same client program to process mailing lists in different formats. With a first-class ADT, the client programs can manipulate the data without direct access, but rather with indirect access, through operations defined in the ADT. An operation such as *extract customer name* then can have different implementations for different list formats. The most important implication of this arrangement is that we can change the data representation without having to change the client programs.

We use the term *handle* to describe a reference to an abstract object. Our goal is to give client programs handles to abstract objects that can be used in assignment statements and as arguments and return values of functions in the same way as built-in data types, while hiding the representation of the objects from the client program.

Program 4.16 is an example of such an interface for complex numbers that achieves this goal, and exemplifies the conventions that we shall use throughout this book. The handle is defined as a pointer to a structure that has a name tag, but is otherwise *not specified*. The client can use this handle as intended, but there can be no code in the client program that uses the handle in any other way: It cannot access a field in a structure by dereferencing the pointer because it does not have the names of any of the fields. In the interface, we define functions which accept handles as arguments and also return handles as values; and client programs can use those functions, all without knowing anything about the data structure that will be used to implement the interface.

Program 4.17 is an implementation of the interface of Program 4.16. It defines the specific data structure that will be used to implement handles and the data type itself; a function that allocates the memory for a new object and initializes its fields; functions that provide access to the fields (which we implement by dereferencing the handle pointer to access the specific fields in the argument objects); and functions that implement the ADT operations. All information specific to the data structure being used is guaranteed to be encapsulated in the implementation, because the client has no way to refer to it.

The distinction between the data type for complex numbers in the code in Programs 4.14 and 4.15 and the ADT for complex numbers in the code in Programs 4.16 and 4.17 is essential and is thus well worth careful study. It is a mechanism that we can use to develop and compare efficient algorithms for fundamental problems throughout this book. We shall not treat all the implications of using such a mechanism for software engineering in further detail, but it is a powerful and general mechanism that will serve us well in the study of algorithms and data structures and their application.

In particular, the issue of storage management is critical in the use of ADTs in software engineering. When we say $x = t$ in Program 4.13, where the variables are both of type `Complex`, we simply are assigning a pointer. The alternative would be to allocate memory for a new object and define an explicit *copy* function to copy the values in the object associated with t to the new object. This issue of *copy semantics* is an important one to address in any ADT design. We normally use pointer assignment (and therefore do not consider *copy* implementations for our ADTs) because of our focus on efficiency—this choice makes us less susceptible to excessive hidden costs when performing operations on huge data structures. The design of the C string data type is based on similar considerations.

The implementation of `COMPLEXmult` in Program 4.15 creates a new object for the result. Alternatively, more in the spirit of reserving explicit object-creation operations for the client, we could return the value in one of the arguments. As it stands, `COMPLEXmult` has a defect called a *memory leak*, that makes the program unusable for a huge number of multiplications. The problem is that each multiplication allocates memory for a new object, but we never execute any calls to `free`. For this reason, ADTs often contain explicit *destroy* operations

Program 4.17 Complex-numbers ADT implementation

By contrast with Program 4.15, this implementation of the complex-numbers ADT includes the structure definition (which is hidden from the client), as well as the function implementations. Objects are pointers to structures, so we dereference the pointer to refer to the fields.

```
#include <stdlib.h>
#include "COMPLEX.h"
struct complex { float Re; float Im; };
Complex COMPLEXinit(float Re, float Im)
{ Complex t = malloc(sizeof *t);
  t->Re = Re; t->Im = Im;
  return t;
}
float Re(Complex z)
{ return z->Re; }
float Im(Complex z)
{ return z->Im; }
Complex COMPLEXmult(Complex a, Complex b)
{
  return COMPLEXinit(Re(a)*Re(b) - Im(a)*Im(b),
                     Re(a)*Im(b) + Im(a)*Re(b));
}
```

for use by clients. However, having the capability for *destroy* is no guarantee that clients will use it for each and every object created, and memory leaks are subtle defects that plague many large systems. For this reason, some programming environments have automatic mechanisms for the system to invoke *destroy*; other systems have *automatic memory allocation*, where the system takes responsibility to figure out which memory is no longer being used by programs, and to reclaim it. None of these solutions is entirely satisfactory. We rarely include *destroy* implementations in our ADTs, since these considerations are somewhat removed from the essential characteristics of our algorithms.

First-class ADTs play a central role in many of our implementations because they provide the necessary support for the abstract mechanisms for generic objects and collections of objects that we dis-

Program 4.18 First-class ADT interface for queues

We provide handles for queues in precisely the same manner as we did for complex numbers in Program 4.16: A handle is a pointer to a structure that is unspecified except for the tag name.

```
typedef struct queue *Q;
void QUEUEDump(Q);
    Q QUEUEInit(int maxN);
    int QUEUEEmpty(Q);
void QUEUEput(Q, Item);
Item QUEUEget(Q);
```

cussed in Section 4.1. Accordingly, we use `Item` for the type of the items that we manipulate in the generalized queue ADTs in this book (and include an `Item.h` interface file), secure in the knowledge that an appropriate implementation will make our code useful for whatever data type a client program might need.

To illustrate further the general nature of the basic mechanism, we consider next a first-class ADT for FIFO queues using the same basic scheme that we just used for complex numbers. Program 4.18 is the interface for this ADT. It differs from Program 4.9 in that it defines a queue handle (to be a pointer to an unspecified structure, in the standard manner) and each function takes a queue handle as an argument. With handles, client programs can manipulate multiple queues.

Program 4.19 is a driver program that exemplifies such a client. It randomly assigns N items to one of M FIFO queues, then prints out the contents of the queues, by removing the items one by one. Figure 4.13 is an example of the output produced by this program. Our interest in this program is to illustrate how the first-class data-type mechanism allows it to work with the queue ADT itself as a high-level object—it could easily be extended to test various methods of organizing queues to serve customers, and so forth.

Program 4.20 is an implementation of the FIFO queue ADT defined in Program 4.18, using linked lists for the underlying data structure. The primary difference between these implementations and those in Program 4.10 has to do with the variables `head` and `tail`.

```
6 13 51 64 71 84 90
4 23 26 34 38 62 78
8 28 33 48 54 56 75 81
2 15 17 37 43 47 50 53 61 80 82
12 25 30 32 36 49 52 63 74 79
3 14 22 27 31 42 46 59 77
9 19 20 29 39 45 69 70 73 76 83
5 11 18 24 35 44 57 58 67
0 1 21 40 41 55 66 72
7 10 16 60 65 68
```

Figure 4.13
Random-queue simulation

This table gives the output that is produced when Program 4.19 is invoked with 84 as the command-line argument. The 10 queues have an average of 8.4 items each, ranging from a low of six to a high of 11.

Program 4.19 Queue client program (queue simulation)

The availability of object handles makes it possible to build compound data structures with ADT objects, such as the array of queues in this sample client program, which simulates a situation where customers waiting for service are assigned at random to one of M service queues.

```
#include <stdio.h>
#include <stdlib.h>
#include "Item.h"
#include "QUEUE.h"
#define M 10
main(int argc, char *argv[])
{ int i, j, N = atoi(argv[1]);
  Q queues[M];
  for (i = 0; i < M; i++)
    queues[i] = QUEUEinit(N);
  for (i = 0; i < N; i++)
    QUEUEput(queues[rand() % M], j);
  for (i = 0; i < M; i++, printf("\n"))
    for (j = 0; !QUEUEempty(queues[i]); j++)
      printf("%3d ", QUEUEget(queues[i]));
}
```

In Program 4.10, we had only one queue, so we simply declared and used these variables in the implementation. In Program 4.20, each queue q has its own pointers `head` and `tail`, which we reference with the code `q->head` and `q->tail`. The definition of `struct queue` in an implementation answers the question “what is a queue?” for that implementation: In this case, the answer is that a queue is pointer to a structure consisting of the links to the head and tail of the queue. In an array implementation, a queue is a pointer to a struct consisting of a pointer to an array and two integers: the size of the array and the number of elements currently on the queue (see Exercise 4.65). In general, the members of the structure are exactly the global or static variables from the one-object implementation.

With a carefully designed ADT, we can make use of the separation between client and implementations in many interesting ways. For example, we commonly use driver programs when developing or

Program 4.20 Linked-list implementation of first-class queue

The code for implementations that provide object handles is typically more cumbersome than the corresponding code for single objects (see Program 4.10). This code does not check for errors such as a client attempt to *get* from an empty queue or an unsuccessful *malloc* (see Exercise 4.33).

```
#include <stdlib.h>
#include "Item.h"
#include "QUEUE.h"
typedef struct QUEUENode* link;
struct QUEUENode { Item item; link next; };
struct queue { link head; link tail; };
link NEW(Item item, link next)
{ link x = malloc(sizeof *x);
  x->item = item; x->next = next;
  return x;
}
Q QUEUEinit(int maxN)
{ Q q = malloc(sizeof *q);
  q->head = NULL;
  return q;
}
int QUEUEempty(Q q)
{ return q->head == NULL; }
void QUEUEput(Q q, Item item)
{
  if (q->head == NULL)
    { q->tail = NEW(item, q->head);
      q->head = q->tail; return; }
  q->tail->next = NEW(item, q->tail->next);
  q->tail = q->tail->next;
}
Item QUEUEget(Q q)
{ Item item = q->head->item;
  link t = q->head->next;
  free(q->head); q->head = t;
  return item;
}
```

debugging ADT implementations. Similarly, we often use incomplete implementations of ADTs, called *stubs*, as placeholders while building systems to learn properties of clients, although this exercise can be tricky for clients that depend on the ADT implementation semantics.

As we saw in Section 4.3, the ability to have multiple instances of a given ADT in a single program can lead us to complicated situations. Do we want to be able to have stacks or queues with different types of objects on them? How about different types of objects on the same queue? Do we want to use different implementations for queues of the same type in a single client because we know of performance differences? Should information about the efficiency of implementations be included in the interface? What form should that information take? Such questions underscore the importance of understanding the basic characteristics of our algorithms and data structures and how client programs may use them effectively, which is, in a sense, the topic of this book. Full implementations, however, are exercises in software engineering, rather than in algorithms design, so we stop short of developing ADTs of such generality in this book (*see reference section*).

Despite its virtues, our mechanism for providing first-class ADTs comes at the (slight) cost of extra pointer dereferences and slightly more complicated implementation code, so we shall use the full mechanism for only those ADTs that require the use of handles as arguments or return values in interfaces. On the one hand, the use of first-class types might encompass the majority of the code in a small number of huge applications systems; on the other hand, an only-one-object arrangement—such as the stacks, FIFO queues, and generalized queues of Sections 4.2 through 4.7—and the use of `typedef` to specify the types of objects as described in Section 4.1 are quite serviceable techniques for many of the programs that we write. In this book, we introduce most of the algorithms and data structures that we consider in the latter context, then extend these implementations into first-class ADTs when warranted.

Exercises

- ▷ 4.57 Add a function `COMPLEXadd` to the ADT for complex numbers in the text (Programs 4.16 and 4.17).
- 4.58 Convert the equivalence-relations ADT in Section 4.5 to a first-class type.
- 4.59 Create a first-class ADT for use in programs that process playing cards.

- 4.60 Write a program to determine empirically the probability that various poker hands are dealt, using your ADT from Exercise 4.59.
- 4.61 Create an ADT for points in the plane, and change the closest-point program in Chapter 3 Program 3.16 to a client program that uses your ADT.
- 4.62 Develop an implementation for the complex-number ADT that is based on representing complex numbers in polar coordinates (that is, in the form $re^{i\theta}$).
- 4.63 Use the identity $e^{i\theta} = \cos \theta + i \sin \theta$ to prove that $e^{2\pi i} = 1$ and that the N complex N th roots of unity are

$$\cos\left(\frac{2\pi k}{N}\right) + i \sin\left(\frac{2\pi k}{N}\right),$$

for $k = 0, 1, \dots, N - 1$.

- 4.64 List the N th roots of unity for N from 2 through 8.
- 4.65 Develop an implementation of the FIFO queue first-class ADT given in the text (Program 4.18) that uses an array as the underlying data structure.
- ▷ 4.66 Write an interface for a first-class pushdown-stack ADT.
- 4.67 Develop an implementation of your first-class pushdown-stack ADT from Exercise 4.66 that uses an array as the underlying data structure.
- 4.68 Develop an implementation of your first-class pushdown-stack ADT from Exercise 4.66 that uses a linked list as the underlying data structure.
- 4.69 Modify the postfix-evaluation program in Section 4.3 to evaluate postfix expressions consisting of complex numbers with integer coefficients, using the first-class complex numbers ADT in the text (Programs 4.16 and 4.17). For simplicity, assume that the complex numbers all have nonnull integer coefficients for both real and imaginary parts and are written with no spaces. For example, your program should print the output 8+4i when given the input
 $1+1i \ 0+1i \ + \ 1-2i \ * \ 3+4i \ + \ .$

4.9 Application-Based ADT Example

As a final example, we consider in this section an application-specific ADT that is representative of the relationship between applications domains and the algorithms and data structures of the type that we consider in this book. The example that we shall consider is the *polynomial* ADT. It is drawn from *symbolic mathematics*, where we use the computer to help us manipulate abstract mathematical objects.

Our goal is to be able to perform computations such as

$$\left(1 - x + \frac{x^2}{2} - \frac{x^3}{6}\right) \left(1 + x + x^2 + x^3\right) = 1 + \frac{x^2}{2} + \frac{x^3}{3} - \frac{2x^4}{3} + \frac{x^5}{3} - \frac{x^6}{6}.$$

Program 4.21 Polynomial client (binomial coefficients)

This client program uses the polynomial ADT that is defined in the interface Program 4.22 to perform algebraic manipulations with polynomials. It takes an integer N and a floating-point number p from the command line, computes $(x + 1)^N$, and checks the result by evaluating the resulting polynomial at $x = p$.

```
#include <stdio.h>
#include <stdlib.h>
#include "POLY.h"
main(int argc, char *argv[])
{ int N = atoi(argv[1]); float p = atof(argv[2]);
  Poly t, x; int i, j;
  printf("Binomial coefficients\n");
  t = POLYadd(POLYterm(1, 1), POLYterm(1, 0));
  for (i = 0, x = t; i < N; i++)
    { x = POLYmult(t, x); showPOLY(x); }
  printf("%f\n", POLYeval(x, p));
}
```

We also want to be able to evaluate the polynomial for a given value of x . For $x = 0.5$, both sides of this equation have the value 1.1328125. The operations of multiplying, adding, and evaluating polynomials are at the heart of a great many mathematical calculations. Program 4.21 is a simple example that performs the symbolic operations corresponding to the polynomial equations

$$\begin{aligned}(x + 1)^2 &= x^2 + 2x + 1, \\(x + 1)^3 &= x^3 + 3x^2 + 3x + 1, \\(x + 1)^4 &= x^4 + 4x^3 + 6x^2 + 4x + 1, \\(x + 1)^5 &= x^5 + 5x^4 + 10x^3 + 10x^2 + 5x + 1, \\&\dots\end{aligned}$$

The same basic ideas extend to include operations such as composition, integration, differentiation, knowledge of special functions, and so forth.

The first step is to define the *polynomial* ADT, as illustrated in the interface Program 4.22. For a well-understood mathematical

abstraction such as a polynomial, the specification is so clear as to be unspoken (in the same way as for the ADT for complex numbers that we discussed in Section 4.8): We want instances of the ADT to behave precisely in the same manner as the well-understood mathematical abstraction.

To implement the functions defined in the interface, we need to choose a particular data structure to represent polynomials and then to implement algorithms that manipulate the data structure to produce the behavior that client programs expect from the ADT. As usual, the choice of data structure affects the potential efficiency of the algorithms, and we are free to consider several. Also as usual, we have the choice of using a linked representation or an array representation. Program 4.23 is an implementation using an array representation; the linked-list representation is left as an exercise (see Exercise 4.70).

To *add* two polynomials, we add their coefficients. If the polynomials are represented as arrays, the *add* function amounts to a single loop through the arrays, as shown in Program 4.23. To *multiply* two polynomials, we use the elementary algorithm based on the distributive law. We multiply one polynomial by each term in the other, line up the results so that powers of x match, then add the terms to get the final result. The following table summarizes the computation for $(1 - x + x^2/2 - x^3/6)(1 + x + x^2 + x^3)$:

$$\begin{array}{r}
 1 - x + \frac{x^2}{2} - \frac{x^3}{6} \\
 + x - x^2 + \frac{x^3}{2} - \frac{x^4}{6} \\
 + x^2 - x^3 + \frac{x^4}{2} - \frac{x^5}{6} \\
 + x^3 - x^4 + \frac{x^5}{2} - \frac{x^6}{6} \\
 \hline
 1 + \frac{x^2}{2} + \frac{x^3}{3} - \frac{2x^4}{3} + \frac{x^5}{3} - \frac{x^6}{6}
 \end{array}$$

The computation seems to require time proportional to N^2 to multiply two polynomials. Finding a faster algorithm for this task is a significant challenge. We shall consider this topic in detail in Part 8, where we shall see that it is possible to accomplish the task in time proportional to

Program 4.22 First-class ADT interface for polynomials

As usual, a handle to a polynomial is a pointer to a structure that is unspecified except for the tag name.

```
typedef struct poly *Poly;
void showPOLY(Poly);
Poly POLYterm(int, int);
Poly POLYadd(Poly, Poly);
Poly POLYmult(Poly, Poly);
float POLYeval(Poly, float);
```

$N^{3/2}$ using a divide-and-conquer algorithm, and in time proportional to $N \lg N$ using the fast Fourier transform.

The implementation of the *evaluate* function in Program 4.23 uses a classic efficient algorithm known as *Horner's algorithm*. A naive implementation of the function involves a direct computation using a function that computes x^N . This approach takes quadratic time. A less naive implementation involves saving the values of x^i in a table, then using them in a direct computation. This approach takes linear extra space. Horner's algorithm is a direct optimal linear algorithm based on parenthesizations such as

$$a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0 = (((a_4x + a_3)x + a_2)x + a_1)x + a_0.$$

Horner's method is often presented as a time-saving trick, but it is actually an early and outstanding example of an elegant and efficient algorithm, which reduces the time required for this essential computational task from quadratic to linear. The calculation that we performed in Program 4.2 for converting ASCII strings to integers is a version of Horner's algorithm. We shall encounter Horner's algorithm again, in Chapter 14 and Part 5, as the basis for an important computation related to certain symbol-table and string-search implementations.

For simplicity and efficiency, POLYadd modifies one of its arguments; if we choose to use this implementation in an application, we should note that fact in the specification (see Exercise 4.71). Moreover, we have memory leaks, particularly in POLYmult, which creates a new polynomial to hold the result (see Exercise 4.72).

As usual, the array representation for implementing the polynomial ADT is but one possibility. If exponents are huge and there are

Program 4.23 Array implementation of polynomial ADT

In this implementation of a first-class ADT for polynomials, a polynomial is a structure containing the degree and a pointer to an array of coefficients. For simplicity in this code, each addition operation modifies one of its arguments and each multiplication operation creates a new object. Another ADT operation to destroy objects (and to free the associated memory) might be needed for some applications.

```
#include <stdlib.h>
#include "POLY.h"
struct poly { int N; int *a; };
Poly POLYterm(int coeff, int exp)
{ int i; Poly t = malloc(sizeof *t);
  t->a = malloc((exp+1)*sizeof(int));
  t->N = exp+1; t->a[exp] = coeff;
  for (i = 0; i < exp; i++) t->a[i] = 0;
  return t;
}
Poly POLYadd(Poly p, Poly q)
{ int i; Poly t;
  if (p->N < q->N) { t = p; p = q; q = t; }
  for (i = 0; i < q->N; i++) p->a[i] += q->a[i];
  return p;
}
Poly POLYmult(Poly p, Poly q)
{ int i, j;
  Poly t = POLYterm(0, (p->N-1)+(q->N-1));
  for (i = 0; i < p->N; i++)
    for (j = 0; j < q->N; j++)
      t->a[i+j] += p->a[i]*q->a[j];
  return t;
}
float POLYeval(Poly p, float x)
{ int i; double t = 0.0;
  for (i = p->N-1; i >= 0; i--)
    t = t*x + p->a[i];
  return t;
}
```

not many terms, a linked-list representation might be more appropriate. For example, we would not want to use Program 4.23 to perform a multiplication such as

$$(1 + x^{1000000})(1 + x^{2000000}) = 1 + x^{1000000} + x^{2000000} + x^{3000000},$$

because it would use an array with space for hundreds of thousands of unused coefficients. Exercise 4.70 explores the linked list option in more detail.

Exercises

- 4.70 Provide an implementation for the polynomial ADT given in the text (Program 4.22) that uses linked lists as the underlying data structure. Your lists should not contain any nodes corresponding to terms with coefficient value 0.
- ▷ 4.71 Modify the implementation of `POLYadd` in Program 4.23 such that it operates in a manner similar to `POLYmult` (and does not modify either of its arguments).
- 4.72 Modify the polynomial ADT interface, implementation, and client in the text (Programs 4.21 through 4.23) such that there are no memory leaks. To do so, define new operations `POLYdestroy` and `POLYcopy`, which should free the memory for an object and copy one object's values to another, respectively; and modify `POLYadd` and `POLYmult` to destroy their arguments and return a newly created object, by convention.
- 4.73 Extend the polynomial ADT given in the text to include integration and differentiation of polynomials.
- 4.74 Modify your polynomial ADT from Exercise 4.73 to ignore all terms with exponents greater than or equal to an integer M , which is provided by the client at initialization.
- 4.75 Extend your polynomial ADT from Exercise 4.73 to include polynomial division and composition.
- 4.76 Develop an ADT that allows clients to perform addition and multiplication of arbitrarily long integers.
- 4.77 Modify the postfix-evaluation program in Section 4.3 to evaluate postfix expressions consisting of arbitrarily long integers, using the ADT that you developed for Exercise 4.76.
- 4.78 Write a client program that uses your polynomial ADT from Exercise 4.75 to evaluate integrals by using Taylor series approximations of functions, manipulating them symbolically.
- 4.79 Develop an ADT that provides clients with the ability to perform algebraic operations on vectors of floating-point numbers.

- 4.80** Develop an ADT that provides clients with the ability to perform algebraic operations on matrices of abstract objects for which addition, subtraction, multiplication, and division are defined.
- 4.81** Write an interface for a character-string ADT, which includes operations for creating a string, comparing two strings, concatenating two strings, copying one string to another, and returning the string length.
- 4.82** Provide an implementation for your string ADT interface from Exercise 4.81, using the C string library where appropriate.
- 4.83** Provide an implementation for your string ADT interface from Exercise 4.81, using a linked list for the underlying representation. Analyze the worst-case running time of each operation.
- 4.84** Write an interface and an implementation for an index set ADT, which processes sets of integers in the range 0 to $M - 1$ (where M is a defined constant) and includes operations for creating a set, computing the union of two sets, computing the intersection of two sets, computing the complement of a set, computing the difference of two sets, and printing out the contents of a set. In your implementation, use an array of $M - 1$ 0-1 values to represent each set.
- 4.85** Write a client program that tests your ADT from Exercise 4.84.

4.10 Perspective

There are three primary reasons for us to be aware of the fundamental concepts underlying ADTs as we embark on the study of algorithms and data structures:

- ADTs are an important software-engineering tool in widespread use, and many of the algorithms that we study serve as implementations for fundamental ADTs that are widely applicable.
- ADTs help us to encapsulate the algorithms that we develop, so that we can use the same code for many different purposes.
- ADTs provide a convenient mechanism for our use in the process of developing and comparing the performance of algorithms.

Ideally, ADTs embody the common-sense principle that we are obligated to describe precisely the ways in which we manipulate our data. The client-interface-implementation mechanism that we have considered in detail in this chapter is convenient for this task in C, and provides us with C code that has a number of desirable properties. Many modern languages have specific support that allows the development of programs with similar properties, but the general approach

transcends particular languages—when we do not have specific language support, we adopt programming conventions to maintain the separation that we would like to have among clients, interfaces, and implementations.

As we consider an ever-expanding set of choices in specifying the behavior of our ADTs, we are faced with an ever-expanding set of challenges in providing efficient implementations. The numerous examples that we have considered illustrate ways of meeting such challenges. We continually strive to achieve the goal of implementing all the operations efficiently, but we are unlikely to have a general-purpose implementation that can do so for all sets of operations. This situation works against the principles that lead us to ADTs in the first place, because in many cases implementors of ADTs need to know properties of client programs to know which implementations of associated ADTs will perform most efficiently, and implementors of client programs need to know performance properties of various implementations to know which to choose for a particular application. As ever, we must strike a balance. In this book, we consider numerous approaches to implementations for variants of fundamental ADTs, all of which have important applications.

We can use one ADT to build another. We have used the pointer and structure abstractions provided by C to build linked lists, then we have used linked lists or the array abstraction provided by C to build pushdown stacks, then we use pushdown stacks to get the capability to evaluate arithmetic expressions. The ADT concept allows us to construct large systems on different layers of abstraction, from the machine-language instructions provided by the computer, to the various capabilities provided by the programming language, to sorting, searching and other higher-level capabilities provided by algorithms as discussed in Parts 3 and 4 of this book, to the even higher levels of abstraction that the various applications require, as discussed in Parts 5 through 8. ADTs are one point on the continuum of developing ever more powerful abstract mechanisms that is the essence of using computers effectively in problem solving.

CHAPTER FIVE

Recursion and Trees

THE CONCEPT OF recursion is fundamental in mathematics and computer science. The simple definition is that a recursive program in a programming language is one that calls itself (just as a recursive function in mathematics is one that is defined in terms of itself). A recursive program cannot call itself always, or it would never stop (just as a recursive function cannot be defined in terms of itself always, or the definition would be circular); so a second essential ingredient is that there must be a *termination condition* when the program can cease to call itself (and when the mathematical function is not defined in terms of itself). All practical computations can be couched in a recursive framework.

The study of recursion is intertwined with the study of recursively defined structures known as *trees*. We use trees both to help us understand and analyze recursive programs and as explicit data structures. We have already encountered an application of trees (although not a recursive one), in Chapter 1. The connection between recursive programs and trees underlies a great deal of the material in this book. We use trees to understand recursive programs; we use recursive programs to build trees; and we draw on the fundamental relationship between both (and recurrence relations) to analyze algorithms. Recursion helps us to develop elegant and efficient data structures and algorithms for all manner of applications.

Our primary purpose in this chapter is to examine recursive programs and data structures as practical tools. First, we discuss the relationship between mathematical recurrences and simple recursive

programs, and we consider a number of examples of practical recursive programs. Next, we examine the fundamental recursive scheme known as *divide and conquer*, which we use to solve fundamental problems in several later sections of this book. Then, we consider a general approach to implementing recursive programs known as *dynamic programming*, which provides effective and elegant solutions to a wide class of problems. Next, we consider trees, their mathematical properties, and associated algorithms in detail, including basic methods for *tree traversal* that underlie recursive tree-processing programs. Finally, we consider closely related algorithms for processing graphs—we look specifically at a fundamental recursive program, *depth-first search*, that serves as the basis for many graph-processing algorithms.

As we shall see, many interesting algorithms are simply expressed with recursive programs, and many algorithm designers prefer to express methods recursively. We also investigate nonrecursive alternatives in detail. Not only can we often devise simple stack-based algorithms that are essentially equivalent to recursive algorithms, but also we can often find nonrecursive alternatives that achieve the same final result through a different sequence of computations. The recursive formulation provides a structure within which we can seek more efficient alternatives.

A full discussion of recursion and trees could fill an entire book, for they arise in many applications throughout computer science, and are pervasive outside of computer science as well. Indeed, it might be said that *this* book is filled with a discussion of recursion and trees, for they are present, in a fundamental way, in every one of the book's chapters.

5.1 Recursive Algorithms

A *recursive algorithm* is one that solves a problem by solving one or more smaller instances of the same problem. To implement recursive algorithms in C, we use *recursive functions*—a recursive function is one that calls itself. Recursive functions in C correspond to recursive definitions of mathematical functions. We begin our study of recursion by examining programs that directly evaluate mathematical functions. The basic mechanisms extend to provide a general-purpose programming paradigm, as we shall see.

Program 5.1 Factorial function (recursive implementation)

This recursive function computes the function $N!$, using the standard recursive definition. It returns the correct value when called with N nonnegative and sufficiently small that $N!$ can be represented as an int.

```
int factorial(int N)
{
    if (N == 0) return 1;
    return N*factorial(N-1);
}
```

Recurrence relations (see Section 2.5) are recursively defined functions. A recurrence relation defines a function whose domain is the nonnegative integers either by some initial values or (recursively) in terms of its own values on smaller integers. Perhaps the most familiar such function is the *factorial* function, which is defined by the recurrence relation

$$N! = N \cdot (N - 1)!, \quad \text{for } N \geq 1 \text{ with } 0! = 1.$$

This definition corresponds directly to the recursive C function in Program 5.1.

Program 5.1 is equivalent to a simple loop. For example, the following for loop performs the same computation:

```
for ( t = 1, i = 1; i <= N; i++) t *= i;
```

As we shall see, it is always possible to transform a recursive program into a nonrecursive one that performs the same computation. Conversely, we can express without loops any computation that involves loops, using recursion, as well.

We use recursion because it often allows us to express complex algorithms in a compact form, without sacrificing efficiency. For example, the recursive implementation of the factorial function obviates the need for local variables. The cost of the recursive implementation is borne by the mechanisms in the programming systems that support function calls, which use the equivalent of a built-in pushdown stack. Most modern programming systems have carefully engineered mechanisms for this task. Despite this advantage, as we shall see, it is all too easy to write a simple recursive function that is extremely inefficient,

Program 5.2 A questionable recursive program

If the argument N is odd, this function calls itself with $3N + 1$ as an argument; if N is even, it calls itself with $N/2$ as an argument. We cannot use induction to prove that this program terminates, because not every recursive call uses an argument smaller than the one given.

```
int puzzle(int N)
{
    if (N == 1) return 1;
    if (N % 2 == 0)
        return puzzle(N/2);
    else return puzzle(3*N+1);
}
```

and we need to exercise care to avoid being burdened with intractable implementations.

Program 5.1 illustrates the basic features of a recursive program: it calls itself (with a smaller value of its argument), and it has a termination condition in which it directly computes its result. We can use mathematical induction to convince ourselves that the program works as intended:

- It computes $0!$ (basis).
- Under the assumption that it computes $k!$ for $k < N$ (inductive hypothesis), it computes $N!$.

Reasoning like this can provide us with a quick path to developing algorithms that solve complex problems, as we shall see.

In a programming language such as C, there are few restrictions on the kinds of programs that we write, but we strive to limit ourselves in our use of recursive functions to those that embody inductive proofs of correctness like the one outlined in the previous paragraph. Although we do not consider formal correctness proofs in this book, we are interested in putting together complicated programs for difficult tasks, and we need to have some assurance that the tasks will be solved properly. Mechanisms such as recursive functions can provide such assurances while giving us compact implementations. Practically speaking, the connection to mathematical induction tells us that we should ensure that our recursive functions satisfy two basic properties:

- They must explicitly solve a basis case.

```
puzzle(3)
  puzzle(10)
    puzzle(5)
      puzzle(16)
        puzzle(8)
          puzzle(4)
            puzzle(2)
              puzzle(1)
```

Figure 5.1
Example of a recursive call chain

This nested sequence of function calls eventually terminates, but we cannot prove that the recursive function in Program 5.2 does not have arbitrarily deep nesting for some argument. We prefer recursive programs that always invoke themselves with smaller arguments.

Program 5.3 Euclid's algorithm

One of the oldest-known algorithms, dating back over 2000 years, is this recursive method for finding the greatest common divisors of two integers.

```
int gcd(int m, int n)
{
    if (n == 0) return m;
    return gcd(n, m % n);
}
```

- Each recursive call must involve smaller values of the arguments. These points are vague—they amount to saying that we should have a valid inductive proof for each recursive function that we write. Still, they provide useful guidance as we develop implementations.

Program 5.2 is an amusing example that illustrates the need for an inductive argument. It is a recursive function that violates the rule that each recursive call must involve smaller values of the arguments, so we cannot use mathematical induction to understand it. Indeed, it is not known whether or not this computation terminates for every N , if there are no bounds on the size of N . For small integers that can be represented as ints, we can check that the program terminates (see Figure 5.1 and Exercise 5.4), but for large integers (64-bit words, say), we do not know whether or not this program goes into an infinite loop.

Program 5.3 is a compact implementation of *Euclid's algorithm* for finding the greatest common divisor of two integers. It is based on the observation that the greatest common divisor of two integers x and y with $x > y$ is the same as the greatest common divisor of y and $x \bmod y$ (the remainder when x is divided by y). A number t divides both x and y if and only if t divides both y and $x \bmod y$, because x is equal to $x \bmod y$ plus a multiple of y . The recursive calls made for an example invocation of this program are shown in Figure 5.2. For Euclid's algorithm, the depth of the recursion depends on arithmetic properties of the arguments (it is known to be logarithmic).

Program 5.4 is an example with multiple recursive calls. It is another expression evaluator, performing essentially the same computations as Program 4.2, but on prefix (rather than postfix) expressions,

```
gcd(314159, 271828)
  gcd(271828, 42331)
    gcd(42331, 17842)
      gcd(17842, 6647)
        gcd(6647, 4458)
          gcd(4458, 2099)
            gcd(2099, 350)
              gcd(350, 349)
                gcd(349, 1)
                  gcd(1, 0)
```

Figure 5.2
Example of Euclid's algorithm

This nested sequence of function calls illustrates the operation of Euclid's algorithm in discovering that 314159 and 271828 are relatively prime.

```

eval() * + 7 * * 4 6 + 8 9 5
  eval() + 7 * * 4 6 + 8 9
    eval() 7
      eval() * * 4 6 + 8 9
        eval() * 4 6
          eval() 4
          eval() 6
          return 24 = 4*6
        eval() + 8 9
          eval() 8
          eval() 9
          return 17 = 8 + 9
        return 408 = 24*17
      return 415 = 7+408
    eval() 5
    return 2075 = 415*5

```

Figure 5.3
Prefix expression evaluation
example

This nested sequence of function calls illustrates the operation of the recursive prefix-expression-evaluation algorithm on a sample expression. For simplicity, the expression arguments are shown here. The algorithm itself never explicitly decides the extent of its argument string; rather, it takes what it needs from the front of the string.

Program 5.4 Recursive program to evaluate prefix expressions

To evaluate a prefix expression, we either convert a number from ASCII to binary (in the while loop at the end), or perform the operation indicated by the first character in the expression on the two operands, evaluated recursively. This function is recursive, but it uses a global array containing the expression and an index to the current character in the expression. The pointer is advanced past each subexpression evaluated.

```

char *a; int i;
int eval()
{ int x = 0;
  while (a[i] == ' ') i++;
  if (a[i] == '+')
    { i++; return eval() + eval(); }
  if (a[i] == '*')
    { i++; return eval() * eval(); }
  while ((a[i] >= '0') && (a[i] <= '9'))
    x = 10*x + (a[i++] - '0');
  return x;
}

```

and letting recursion take the place of the explicit pushdown stack. In this chapter, we shall see many other examples of recursive programs and equivalent programs that use pushdown stacks. We shall examine the specific relationship between several pairs of such programs in detail.

Figure 5.3 shows the operation of Program 5.4 on a sample prefix expression. The multiple recursive calls mask a complex series of computations. Like most recursive programs, this program is best understood inductively: Assuming that it works properly for simple expressions, we can convince ourselves that it works properly for complex ones. This program is a simple example of a *recursive descent parser*—we can use the same process to convert C programs into machine code.

A precise inductive proof that Program 5.4 evaluates the expression properly is certainly much more challenging to write than are the proofs for functions with integer arguments that we have been

discussing, and we shall encounter recursive programs and data structures that are even more complicated than this one throughout this book. Accordingly, we do not pursue the idealistic goal of providing complete inductive proofs of correctness for every recursive program that we write. In this case, the ability of the program to “know” how to separate the operands corresponding to a given operator seems mysterious at first (perhaps because we cannot immediately see how to do this separation at the top level), but is actually a straightforward calculation (because the path to pursue at each function call is unambiguously determined by the first character in the expression).

In principle, we can replace any `for` loop by an equivalent recursive program. Often, the recursive program is a more natural way to express the computation than the `for` loop, so we may as well take advantage of the mechanism provided by the programming system that supports recursion. There is one hidden cost, however, that we need to bear in mind. As is plain from the examples that we examined in Figures 5.1 through 5.3, when we execute a recursive program, we are nesting function calls, until we reach a point where we do not do a recursive call, and we return instead. In most programming environments, such nested function calls are implemented using the equivalent of built-in pushdown stacks. We shall examine the nature of such implementations throughout this chapter. The *depth of the recursion* is the maximum degree of nesting of the function calls over the course of the computation. Generally, the depth will depend on the input. For example, the depths of the recursions for the examples depicted in Figures 5.2 and 5.3 are 9 and 4, respectively. When using a recursive program, we need to take into account that the programming environment has to maintain a pushdown stack of size proportional to the depth of the recursion. For huge problems, the space needed for this stack might prevent us from using a recursive solution.

Data structures built from nodes with pointers are inherently recursive. For example, our definition of linked lists in Chapter 3 (Definition 3.3) is recursive. Therefore, recursive programs provide natural implementations of many commonly used functions for manipulating such data structures. Program 5.5 comprises four examples. We use such implementations frequently throughout the book, primarily because they are so much easier to understand than are their nonrecursive counterparts. However, we must exercise caution in using programs

such as those in Program 5.5 when processing huge lists, because the depth of the recursion for those functions can be proportional to the length of the lists, so the space required for the recursive stack might become prohibitive.

Some programming environments automatically detect and eliminate *tail recursion*, when the last action of a function is a recursive call, because it is not strictly necessary to add to the depth of the recursion in such a case. This improvement would effectively transform the count, traversal, and deletion functions in Program 5.5 into loops, but it does not apply to the reverse-order traversal function.

In Sections 5.2 and 5.3, we consider two families of recursive algorithms that represent essential computational paradigms. Then, in Sections 5.4 through 5.7, we consider recursive data structures that serve as the basis for a very large fraction of the algorithms that we consider.

Exercises

- ▷ 5.1 Write a recursive program to compute $\lg(N!)$.
- 5.2 Modify Program 5.1 to compute $N! \bmod M$, such that overflow is no longer an issue. Try running your program for $M = 997$ and $N = 10^3, 10^4, 10^5$, and 10^6 , to get an indication of how your programming system handles deeply nested recursive calls.
- ▷ 5.3 Give the sequences of argument values that result when Program 5.2 is invoked for each of the integers 1 through 9.
- 5.4 Find the value of $N < 10^6$ for which Program 5.2 makes the maximum number of recursive calls.
- ▷ 5.5 Provide a nonrecursive implementation of Euclid's algorithm.
- ▷ 5.6 Give the figure corresponding to Figure 5.2 for the result of running Euclid's algorithm for the inputs 89 and 55.
- 5.7 Give the recursive depth of Euclid's algorithm when the input values are two consecutive Fibonacci numbers (F_N and F_{N+1}).
- ▷ 5.8 Give the figure corresponding to Figure 5.3 for the result of recursive prefix-expression evaluation for the input $+ * * 12\ 12\ 12\ 144$.
- 5.9 Write a recursive program to evaluate postfix expressions.
- 5.10 Write a recursive program to evaluate infix expressions. You may assume that operands are always enclosed in parentheses.
- 5.11 Write a recursive program that converts infix expressions to postfix.
- 5.12 Write a recursive program that converts postfix expressions to infix.

Program 5.5 Examples of recursive functions for linked lists

These recursive functions for simple list-processing tasks are easy to express, but may not be useful for huge lists because the depth of the recursion may be proportional to the length of the list.

The first function, `count`, counts the number of nodes on the list. The second, `traverse`, calls the function `visit` for each node on the list, from beginning to end. These two functions are both also easy to implement with a `for` or `while` loop. The third function, `traverseR`, does not have a simple iterative counterpart. It calls the function `visit` for every node on the list, but in reverse order.

The fourth function, `delete`, makes the structural changes needed for a given item to be deleted from a list. It returns a link to the (possibly altered) remainder of the list—the link returned is `x`, except when `x->item` is `v`, when the link returned is `x->next` (and the recursion stops).

```
int count(link x)
{
    if (x == NULL) return 0;
    return 1 + count(x->next);
}

void traverse(link h, void (*visit)(link))
{
    if (h == NULL) return;
    (*visit)(h);
    traverse(h->next, visit);
}

void traverseR(link h, void (*visit)(link))
{
    if (h == NULL) return;
    traverseR(h->next, visit);
    (*visit)(h);
}

link delete(link x, Item v)
{
    if (x == NULL) return NULL;
    if (eq(x->item, v))
        { link t = x->next; free(x); return t; }
    x->next = delete(x->next, v);
    return x;
}
```

- 5.13 Write a recursive program for solving the Josephus problem (see Section 3.3).
- 5.14 Write a recursive program that deletes the final element of a linked list.
- 5.15 Write a recursive program for reversing the order of the nodes in a linked list (see Program 3.7). *Hint:* Use a global variable.

5.2 Divide and Conquer

Many of the recursive programs that we consider in this book use two recursive calls, each operating on about one-half of the input. This recursive scheme is perhaps the most important instance of the well-known *divide-and-conquer* paradigm for algorithm design, which serves as the basis for many of our most important algorithms.

As an example, let us consider the task of finding the maximum among N items stored in an array $a[0], \dots, a[N-1]$. We can easily accomplish this task with a single pass through the array, as follows:

```
for (t = a[0], i = 1; i < N; i++)
    if (a[i] > t) t = a[i];
```

The recursive divide-and-conquer solution given in Program 5.6 is also a simple (entirely different) algorithm for the same problem; we use it to illustrate the divide-and-conquer concept.

Most often, we use the divide-and-conquer approach because it provides solutions faster than those available with simple iterative algorithms (we shall discuss several examples at the end of this section), but it also is worthy of close examination as a way of understanding the nature of certain fundamental computations.

Figure 5.4 shows the recursive calls that are made when Program 5.6 is invoked for a sample array. The underlying structure seems complicated, but we normally do not need to worry about it—we depend on a proof by induction that the program works, and we use a recurrence relation to analyze the program's performance.

As usual, the code itself suggests the proof by induction that it performs the desired computation:

- It finds the maximum for arrays of size 1 explicitly and immediately.
- For $N > 1$, it partitions the array into two arrays of size less than N , finds the maximum of the two parts by the inductive

0 1 2 3 4 5 6 7 8 9 10
T I N Y E X A M P L E

```
Y max(0, 10)
  Y max(0, 5)
    T max(0, 2)
      T max(0, 1)
        T max(0, 0)
        I max(1, 1)
      N max(2, 2)
    Y max(3, 5)
      Y max(3, 4)
        Y max(3, 3)
        E max(4, 4)
      X max(5, 5)
    P max(6, 10)
      P max(6, 8)
        M max(6, 7)
          A max(6, 6)
          M max(7, 7)
        P max(8, 8)
      L max(9, 10)
        L max(9, 9)
        E max(10, 10)
```

Figure 5.4
A recursive approach to finding the maximum

This sequence of function calls illustrates the dynamics of finding the maximum with a recursive algorithm.

Program 5.6 Divide-and-conquer to find the maximum

This function divides a file $a[l], \dots, a[r]$ into $a[l], \dots, a[m]$ and $a[m+1], \dots, a[r]$, finds the maximum elements in the two parts (recursively), and returns the larger of the two as the maximum element in the whole file. It assumes that `Item` is a first-class type for which `>` is defined. If the file size is even, the two parts are equal in size; if the file size is odd, the size of the first part is 1 greater than the size of the second part.

```
Item max(Item a[], int l, int r)
{ Item u, v; int m = (l+r)/2;
  if (l == r) return a[l];
  u = max(a, l, m);
  v = max(a, m+1, r);
  if (u > v) return u; else return v;
}
```

hypothesis, and returns the larger of these two values, which must be the maximum value in the whole array.

Moreover, we can use the recursive structure of the program to understand its performance characteristics.

Property 5.1 *A recursive function that divides a problem of size N into two independent (nonempty) parts that it solves recursively calls itself less than N times.*

If the parts are one of size k and one of size $N - k$, then the total number of recursive function calls that we use is

$$T_N = T_k + T_{N-k} + 1, \quad \text{for } N \geq 1 \text{ with } T_1 = 0.$$

The solution $T_N = N - 1$ is immediate by induction. If the sizes sum to a value less than N , the proof that the number of calls is less than $N - 1$ follows the same inductive argument. We can prove analogous results under general conditions (see Exercise 5.20). ■

Program 5.6 is representative of many divide-and-conquer algorithms with precisely the same recursive structure, but other examples may differ in two primary respects. First, Program 5.6 does a constant amount of work on each function call, so its total running time is linear. Other divide-and-conquer algorithms may perform more work

```

0 10
0 5 6 10
0 2 3 5 6 10
0 1 2 2 3 5 6 10
0 0 1 1 2 2 3 5 6 10
1 1 2 2 3 5 6 10
2 2 3 5 6 10
3 5 6 10
3 4 5 5 6 10
3 3 4 4 5 5 6 10
4 4 5 5 6 10
5 5 6 10
6 10
6 8 9 10
6 7 8 8 9 10
6 6 7 7 8 8 9 10
7 7 8 8 9 10
8 8 9 10
9 10
9 9 10 10
10 10
```

Figure 5.5
Example of internal stack dynamics

This sequence is an idealistic representation of the contents of the internal stack during the sample computation of Figure 5.4. We start with the left and right indices of the whole subarray on the stack. Each line depicts the result of popping two indices and, if they are not equal, pushing four indices, which delimit the left subarray and the right subarray after the popped subarray is divided into two parts. In practice, the system keeps return addresses and local variables on the stack, instead of this specific representation of the work to be done, but this model suffices to describe the computation.

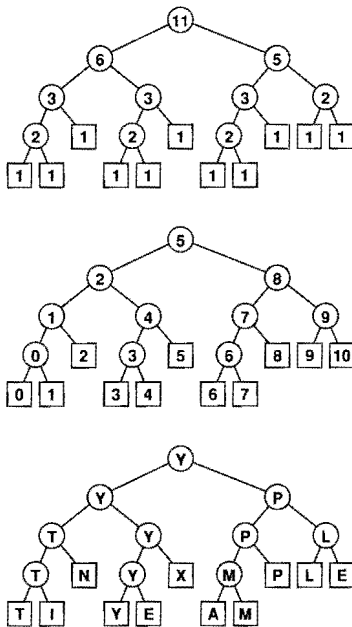


Figure 5.6
Recursive structure of find-the-maximum algorithm.

The divide-and-conquer algorithm splits a problem of size 11 into one of size 6 and one of size 5, a problem of size 6 into two problems of size 3, and so forth, until reaching problems of size 1 (top). Each circle in these diagrams represents a call on the recursive function, to the nodes just below connected to it by lines (squares are those calls for which the recursion terminates). The diagram in the middle shows the value of the index into the middle of the file that we use to effect the split; the diagram at the bottom shows the return value.

on each function call, as we shall see, so determining the total running time requires more intricate analysis. The running time of such algorithms depends on the precise manner of division into parts. Second, Program 5.6 is representative of divide-and-conquer algorithms for which the parts sum to make the whole. Other divide-and-conquer algorithms may divide into smaller parts that constitute less than the whole problem, or overlapping parts that total up to more than the whole problem. These algorithms are still proper recursive algorithms because *each* part is smaller than the whole, but analyzing them is more difficult than analyzing Program 5.6. We shall consider the analysis of these different types of algorithms in detail as we encounter them.

For example, the binary-search algorithm that we studied in Section 2.6 is a divide-and-conquer algorithm that divides a problem in half, then works on just one of the halves. We examine a recursive implementation of binary search in Chapter 12.

Figure 5.5 indicates the contents of the internal stack maintained by the programming environment to support the computation in Figure 5.4. The model depicted in the figure is idealistic, but it gives useful insights into the structure of the divide-and-conquer computation. If a program has two recursive calls, the actual internal stack contains one entry corresponding to the first function call while that function is being executed (which contains values of arguments, local variables, and a return address), then a similar entry corresponding to the second function call while that function is being executed. The alternative that is depicted in Figure 5.5 is to put the two entries on the stack at once, keeping all the subtasks remaining to be done explicitly on the stack. This arrangement plainly delineates the computation, and sets the stage for more general computational schemes, such as those that we examine in Sections 5.6 and 5.8.

Figure 5.6 depicts the structure of the divide-and-conquer find-the-maximum computation. It is a recursive structure: the node at the top contains the size of the input array, the structure for the left subarray is drawn at the left and the structure for the right subarray is drawn at the right. We will formally define and discuss tree structures of this type in Sections 5.4 and 5.5. They are useful for understanding the structure of any program involving nested function calls—recursive programs in particular. Also shown in Figure 5.6 is the same tree, but with each node labeled with the return value for the corresponding

Program 5.7 Solution to the towers of Hanoi

We shift the tower of disks to the right by (recursively) shifting all but the bottom disk to the left, then shifting the bottom disk to the right, then (recursively) shifting the tower back onto the bottom disk.

```
void hanoi(int N, int d)
{
    if (N == 0) return;
    hanoi(N-1, -d);
    shift(N, d);
    hanoi(N-1, -d);
}
```

function call. In Section 5.7, we shall consider the process of building explicit linked structures that represent trees like this one.

No discussion of recursion would be complete without the ancient *towers of Hanoi* problem. We have three pegs and N disks that fit onto the pegs. The disks differ in size, and are initially arranged on one of the pegs, in order from largest (disk N) at the bottom to smallest (disk 1) at the top. The task is to move the stack of disks to the right one position (peg), while obeying the following rules: (i) only one disk may be shifted at a time; and (ii) no disk may be placed on top of a smaller one. One legend says that the world will end when a certain group of monks accomplishes this task in a temple with 40 golden disks on three diamond pegs.

Program 5.7 gives a recursive solution to the problem. It specifies which disk should be shifted at each step, and in which direction (+ means move one peg to the right, cycling to the leftmost peg when on the rightmost peg; and - means move one peg to the left, cycling to the rightmost peg when on the leftmost peg). The recursion is based on the following idea: To move N disks one peg to the right, we first move the top $N - 1$ disks one peg to the left, then shift disk N one peg to the right, then move the $N - 1$ disks one more peg to the left (onto disk N). We can verify that this solution works by induction. Figure 5.7 shows the moves for $N = 5$ and the recursive calls for $N = 3$. An underlying pattern is evident, which we now consider in detail.

First, the recursive structure of this solution immediately tells us the number of moves that the solution requires.

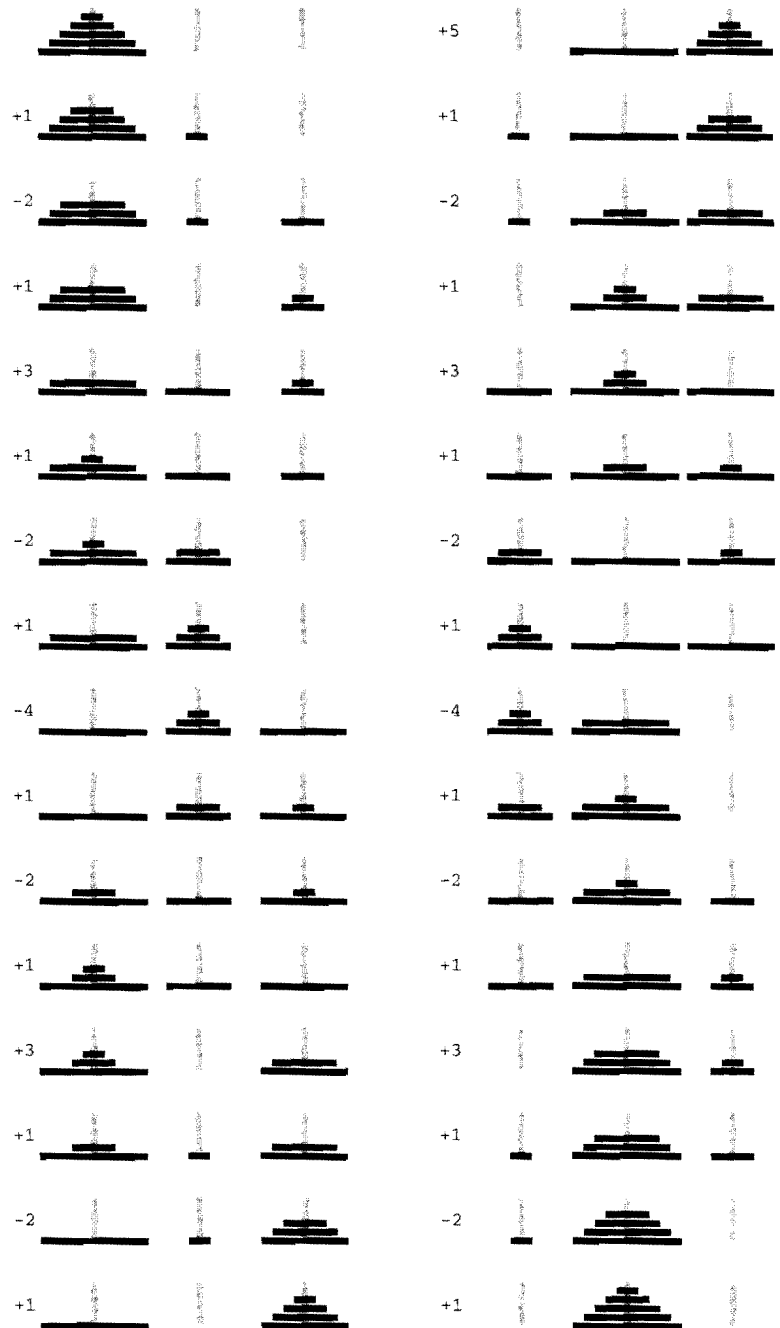
Figure 5.7
Towers of Hanoi

This diagram depicts the solution to the towers of Hanoi problem for five disks. We shift the top four disks left one position (left column), then move disk 5 to the right, then shift the top four disks left one position (right column). The sequence of function calls that follows constitutes the computation for three disks. The computed sequence of moves is +1 -2 +1 +3 +1 -2 +1, which appears four times in the solution (for example, the first seven moves).

```

hanoi(3, +1)
  hanoi(2, -1)
    hanoi(1, +1)
      hanoi(0, -1)
      shift(1, +1)
      hanoi(0, -1)
    shift(2, -1)
    hanoi(1, +1)
      hanoi(0, -1)
      shift(1, +1)
      hanoi(0, -1)
  shift(3, +1)
  hanoi(2, -1)
    hanoi(1, +1)
      hanoi(0, -1)
      shift(1, +1)
      hanoi(0, -1)
    shift(2, -1)
    hanoi(1, +1)
      hanoi(0, -1)
      shift(1, +1)
      hanoi(0, -1)

```



Property 5.2 *The recursive divide-and-conquer algorithm for the towers of Hanoi problem produces a solution that has $2^N - 1$ moves.*

As usual, it is immediate from the code that the number of moves satisfies a recurrence. In this case, the recurrence satisfied by the number of disk moves is similar to Formula 2.5:

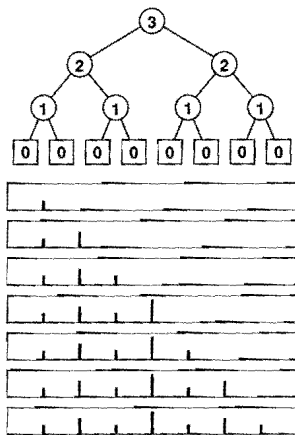
$$T_N = 2T_{N-1} + 1, \quad \text{for } N \geq 2 \text{ with } T_1 = 1.$$

We can verify the stated result directly by induction: we have $T(1) = 2^1 - 1 = 1$; and, if $T(k) = 2^k - 1$ for $k < N$, then $T(N) = 2(2^{N-1} - 1) + 1 = 2^N - 1$. ■

If the monks are moving disks at the rate of one per second, it will take at least 348 centuries for them to finish (see Figure 2.1), assuming that they do not make a mistake. The end of the world is likely be even further off than that because those monks presumably never have had the benefit of being able to use Program 5.7, and might not be able to figure out so quickly which disk to move next. We now consider an analysis of the method that leads to a simple (nonrecursive) method that makes the decision easy. While we may not wish to let the monks in on the secret, it is relevant to numerous important practical algorithms.

To understand the towers of Hanoi solution, let us consider the simple task of drawing the markings on a ruler. Each inch on the ruler has a mark at the 1/2 inch point, slightly shorter marks at 1/4 inch intervals, still shorter marks at 1/8 inch intervals, and so forth. Our task is to write a program to draw these marks at any given resolution, assuming that we have at our disposal a procedure `mark(x, h)` to make a mark h units high at position x .

If the desired resolution is $1/2^n$ inches, we rescale so that our task is to put a mark at every point between 0 and 2^n , endpoints not included. Thus, the middle mark should be n units high, the marks in the middle of the left and right halves should be $n - 1$ units high, and so forth. Program 5.8 is a straightforward divide-and-conquer algorithm to accomplish this objective; Figure 5.8 illustrates it in operation on a small example. Recursively speaking, the idea behind the method is the following. To make the marks in an interval, we first divide the interval into two equal halves. Then, we make the (shorter) marks in the left half (recursively), the long mark in the middle, and the (shorter) marks in the right half (recursively). Iteratively speaking, Figure 5.8



```

rule(0, 8, 3)
  rule(0, 4, 2)
    rule(0, 2, 1)
      rule(0, 1, 0)
        mark(1, 1)
      rule(1, 2, 0)
        mark(2, 2)
      rule(2, 4, 1)
        rule(2, 3, 0)
          mark(3, 1)
        rule(3, 4, 0)
          mark(4, 3)
        rule(4, 8, 2)
          rule(4, 6, 1)
            rule(4, 5, 0)
              mark(5, 1)
            rule(5, 6, 0)
              mark(6, 2)
            rule(6, 8, 1)
              rule(6, 7, 0)
                mark(7, 1)
              rule(7, 8, 0)

```

Figure 5.8
Ruler-drawing function calls

This sequence of function calls constitutes the computation for drawing a ruler of length 8, resulting in marks of lengths 1, 2, 1, 3, 1, 2, and 1.

Program 5.8 Divide and conquer to draw a ruler

To draw the marks on a ruler, we draw the marks on the left half, then draw the longest mark in the middle, then draw the marks on the right half. This program is intended to be used with $r - l$ equal to a power of 2—a property that it preserves in its recursive calls (see Exercise 5.27).

```

rule(int l, int r, int h)
{
    int m = (l+r)/2;
    if (h > 0)
    {
        rule(l, m, h-1);
        mark(m, h);
        rule(m, r, h-1);
    }
}

```

illustrates that the method makes the marks in order, from left to right—the trick lies in computing the lengths. The recursion tree in the figure helps us to understand the computation: Reading down, we see that the length of the mark decreases by 1 for each recursive function call. Reading across, we get the marks in the order that they are drawn, because, for any given node, we first draw the marks associated with the function call on the left, then the mark associated with the node, then the marks associated with the function call on the right.

We see immediately that the sequence of lengths is precisely the same as the sequence of disks moved for the towers of Hanoi problem. Indeed, a simple proof that they are identical is that the recursive programs are the same. Put another way, our monks could use the marks on a ruler to decide which disk to move.

Moreover, both the towers of Hanoi solution in Program 5.7 and the ruler-drawing program in Program 5.8 are variants of the basic divide-and-conquer scheme exemplified by Program 5.6. All three solve a problem of size 2^n by dividing it into two problems of size 2^{n-1} . For finding the maximum, we have a linear-time solution in the size of the input; for drawing a ruler and for solving the towers of Hanoi, we have a linear-time solution in the size of the output. For the towers of Hanoi, we normally think of the solution as being

exponential time, because we measure the size of the problem in terms of the number of disks, n .

It is easy to draw the marks on a ruler with a recursive program, but is there some simpler way to compute the length of the i th mark, for any given i ? Figure 5.9 shows yet another simple computational process that provides the answer to this question. The i th number printed out by both the towers of Hanoi program and the ruler program is nothing other than the number of trailing 0 bits in the binary representation of i . We can prove this property by induction by correspondence with a divide-and-conquer formulation for the process of printing the table of n -bit numbers: Print the table of $(n - 1)$ -bit numbers, each preceded by a 0 bit, then print the table of $(n - 1)$ -bit numbers each preceded by a 1-bit (see Exercise 5.25).

For the towers of Hanoi problem, the implication of the correspondence with n -bit numbers is a simple algorithm for the task. We can move the pile one peg to the right by iterating the following two steps until done:

- Move the small disk to the right if n is odd (left if n is even).
- Make the only legal move not involving the small disk.

That is, after we move the small disk, the other two pegs contain two disks, one smaller than the other. The only legal move not involving the small disk is to move the smaller one onto the larger one. Every other move involves the small disk for the same reason that every other number is odd and that every other mark on the ruler is the shortest. Perhaps our monks *do* know this secret, because it is hard to imagine how they might be deciding which moves to make otherwise.

A formal proof by induction that every other move in the towers of Hanoi solution involves the small disk (beginning and ending with such moves) is instructive: For $n = 1$, there is just one move, involving the small disk, so the property holds. For $n > 1$, the assumption that the property holds for $n - 1$ implies that it holds for n by the recursive construction: The first solution for $n - 1$ begins with a small-disk move, and the second solution for $n - 1$ ends with a small-disk move, so the solution for n begins and ends with a small-disk move. We put a move not involving the small disk in between two moves that do involve the small disk (the move ending the first solution for $n - 1$ and the move beginning the second solution for $n - 1$), so the property that every other move involves the small disk is preserved.

0	0	0	0	1	
0	0	0	1	0	1
0	0	0	1	1	
0	0	1	0	0	2
0	0	1	0	1	
0	0	1	1	0	1
0	0	1	1	1	
0	1	0	0	0	3
0	1	0	0	1	
0	1	0	1	0	1
0	1	0	1	1	
0	1	1	0	0	2
0	1	1	0	1	
0	1	1	1	0	1
0	1	1	1	1	
1	0	0	0	0	4
1	0	0	0	1	
1	0	0	1	0	1
1	0	0	1	1	
1	0	1	0	0	2
1	0	1	0	1	
1	0	1	1	0	1
1	0	1	1	1	
1	1	0	0	0	3
1	1	0	0	1	
1	1	0	1	0	1
1	1	0	1	1	
1	1	1	0	0	2
1	1	1	0	1	
1	1	1	1	0	1
1	1	1	1	1	

Figure 5.9
Binary counting and the ruler function

Computing the ruler function is equivalent to counting the number of trailing zeros in the even N -bit numbers.

Program 5.9 Nonrecursive program to draw a ruler

In contrast to Program 5.8, we can also draw a ruler by first drawing all the marks of length 1, then drawing all the marks of length 2, and so forth. The variable t carries the length of the marks and the variable j carries the number of marks in between two successive marks of length t . The outer for loop increments t and preserves the property $j = 2^{t-1}$. The inner for loop draws all the marks of length t .

```
rule(int l, int r, int h)
{
    int i, j, t;
    for (t = 1, j = 1; t <= h; j += j, t++)
        for (i = 0; l+j+i <= r; i += j+j)
            mark(l+j+i, t);
}
```

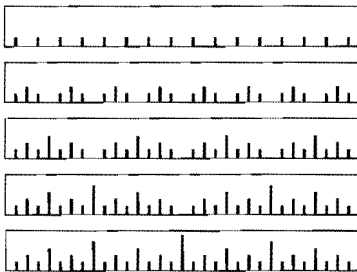


Figure 5.10
Drawing a ruler in bottom-up order

To draw a ruler nonrecursively, we alternate drawing marks of length 1 and skipping positions, then alternate drawing marks of length 2 and skipping remaining positions, then alternate drawing marks of length 3 and skipping remaining positions, and so forth.

Program 5.9 is an alternate way to draw a ruler that is inspired by the correspondence to binary numbers (see Figure 5.10). We refer to this version of the algorithm as a *bottom-up* implementation. It is not recursive, but it is certainly suggested by the recursive algorithm. This correspondence between divide-and-conquer algorithms and the binary representations of numbers often provides insights for analysis and development of improved versions, such as bottom-up approaches. We consider this perspective to understand, and possibly to improve, each of the divide-and-conquer algorithms that we examine.

The bottom-up approach involves rearranging the *order* of the computation when we are drawing a ruler. Figure 5.11 shows another example, where we rearrange the order of the three function calls in the recursive implementation. It reflects the recursive computation in the way that we first described it: Draw the middle mark, then draw the left half, then draw the right half. The pattern of drawing the marks is complex, but is the result of simply exchanging two statements in Program 5.8. As we shall see in Section 5.6, the relationship between Figures 5.8 and 5.11 is akin to the distinction between postfix and prefix in arithmetic expressions.

Drawing the marks in order as in Figure 5.8 might be preferable to doing the rearranged computations contained in Program 5.9 and indicated in Figure 5.11, because we can draw an arbitrarily long

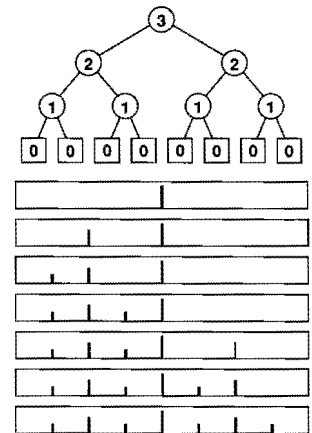
ruler, if we imagine a drawing device that simply moves on to the next mark in a continuous scroll. Similarly, to solve the towers of Hanoi problem, we are constrained to produce the sequence of disk moves in the order that they are to be performed. In general, many recursive programs depend on the subproblems being solved in a particular order. For other computations (see, for example, Program 5.6), the order in which we solve the subproblems is irrelevant. For such computations, the only constraint is that we must solve the subproblems before we can solve the main problem. Understanding when we have the flexibility to reorder the computation not only is a secret to success in algorithm design, but also has direct practical effects in many contexts. For example, this matter is critical when we consider implementing algorithms on parallel processors.

The bottom-up approach corresponds to the general method of algorithm design where we solve a problem by first solving trivial subproblems, then combining those solutions to solve slightly bigger subproblems, and so forth, until the whole problem is solved. This approach might be called *combine and conquer*.

It is a small step from drawing rulers to drawing two-dimensional patterns such as Figure 5.12. This figure illustrates how a simple recursive description can lead to a computation that appears to be complex (see Exercise 5.30).

Recursively defined geometric patterns such as Figure 5.12 are sometimes called *fractals*. If more complicated drawing primitives are used, and more complicated recursive invocations are involved (especially including recursively-defined functions on reals and in the complex plane), patterns of remarkable diversity and complexity can be developed. Another example, demonstrated in Figure 5.13, is the *Koch star*, which is defined recursively as follows: A Koch star of order 0 is the simple hill example of Figure 4.3, and a Koch star of order n is a Koch star of order $n - 1$ with each line segment replaced by the star of order 0, scaled appropriately.

Like the ruler-drawing and the towers of Hanoi solutions, these algorithms are linear in the number of steps, but that number is exponential in the maximum depth of the recursion (see Exercises 5.29 and 5.33). They also can be directly related to counting in an appropriate number system (see Exercise 5.34).



```
rule(0, 8, 3)
  mark(4, 3)
  rule(0, 4, 2)
    mark(2, 2)
    rule(0, 2, 1)
      mark(1, 1)
      rule(0, 1, 0)
      rule(1, 2, 0)
    rule(2, 4, 1)
      mark(3, 1)
      rule(2, 3, 0)
      rule(3, 4, 0)
  rule(4, 8, 2)
    mark(6, 2)
    rule(4, 6, 1)
      mark(5, 1)
      rule(4, 5, 0)
      rule(5, 6, 0)
    rule(6, 8, 1)
      mark(7, 1)
      rule(6, 7, 0)
      rule(7, 8, 0)
```

Figure 5.11
Ruler-drawing function calls
(preorder version)

This sequence indicates the result of drawing marks before the recursive calls, instead of in between them.

Table 5.1 Basic divide-and-conquer algorithms

Binary search (see Chapters 2 and 12) and mergesort (see Chapter 8) are prototypical divide-and-conquer algorithms that provide guaranteed optimal performance for searching and sorting, respectively. The recurrences indicate the nature of the divide-and-conquer computation for each algorithm. (See Sections 2.5 and 2.6 for derivations of the solutions in the rightmost column.) Binary search splits a problem in half, does 1 comparison, then makes a recursive call for one of the halves. Mergesort splits a problem in half, then works on both halves recursively, then does N comparisons. Throughout the book, we shall consider numerous other algorithms developed with these recursive schemes.

	recurrence	approximate solution
binary search		
comparisons	$C_N = C_{N/2} + 1$	$\lg N$
mergesort		
recursive calls	$A_N = 2A_{N/2} + 1$	N
comparisons	$C_N = 2C_{N/2} + N$	$N \lg N$

The towers of Hanoi problem, ruler-drawing problem, and fractals are amusing; and the connection to binary numbers is surprising, but our primary interest in all of these topics is that they provide us with insights in understanding the basic algorithm design paradigm of divide in half and solve one or both halves independently, which is perhaps the most important such technique that we consider in this book. Table 5.1 includes details about binary search and mergesort, which not only are important and widely used practical algorithms, but also exemplify the divide-and-conquer algorithm design paradigm.

Quicksort (see Chapter 7) and binary-tree search (see Chapter 12) represent a significant variation on the basic divide-and-conquer theme where the problem is split into subproblems of size $k - 1$ and $N - k$, for some value k , which is determined by the input. For random input, these algorithms divide a problem into subproblems that are half the size (as in mergesort or in binary search) *on the average*. We study the analysis of the effects of this difference when we discuss these algorithms.

Other variations on the basic theme that are worthy of consideration include these: divide into parts of varying size, divide into more than two parts, divide into overlapping parts, and do various amounts of work in the nonrecursive part of the algorithm. In general, divide-and-conquer algorithms involve doing work to split the input into pieces, or to merge the results of processing two independent solved portions of the input, or to help things along after half of the input has been processed. That is, there may be code before, after, or in between the two recursive calls. Naturally, such variations lead to algorithms more complicated than are binary search and mergesort, and are more difficult to analyze. We consider numerous examples in this book; we return to advanced applications and analysis in Part 8.

Exercises

- 5.16 Write a recursive program that finds the maximum element in an array, based on comparing the first element in the array against the maximum element in the rest of the array (computed recursively).
- 5.17 Write a recursive program that finds the maximum element in a linked list.
- 5.18 Modify the divide-and-conquer program for finding the maximum element in an array (Program 5.6) to divide an array of size N into one part of size $k = 2^{\lceil \lg N \rceil - 1}$ and another of size $N - k$ (so that the size of at least one of the parts is a power of 2).
- 5.19 Draw the tree corresponding to the recursive calls that your program from Exercise 5.18 makes when the array size is 11.
- 5.20 Prove by induction that the number of function calls made by any divide-and-conquer algorithm that divides a problem into parts that constitute the whole, then solves the parts recursively, is linear.
- 5.21 Prove that the recursive solution to the towers of Hanoi problem (Program 5.7) is optimal. That is, show that any solution *requires* at least $2^N - 1$ moves.
- ▷ 5.22 Write a recursive program that computes the length of the i th mark in a ruler with $2^n - 1$ marks.
- 5.23 Examine tables of n -bit numbers, such as Figure 5.9, to discover a property of the i th number that determines the direction of the i th move (indicated by the sign bit in Figure 5.7) for solving the towers of Hanoi problem.
- 5.24 Write a program that produces a solution to the towers of Hanoi problem by filling in an array that holds all the moves, as in Program 5.9.

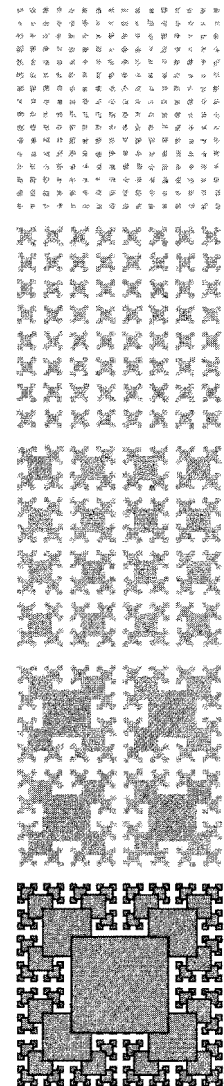
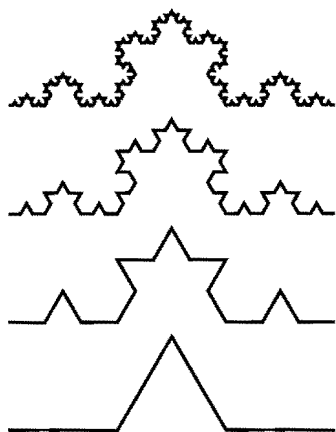


Figure 5.12
Two-dimensional fractal star

This fractal is a two-dimensional version of Figure 5.10. The out-lined boxes in the bottom diagram highlight the recursive structure of the computation.



```

/kochR
{
  2 copy ge {dup 0 rlineto }
  {
    3 div
    2 copy kochR
    60 rotate
    2 copy kochR
    -120 rotate
    2 copy kochR
    60 rotate
    2 copy kochR
  } ifelse
  pop pop
} def
0 0 moveto
27 81 kochR
0 27 moveto
9 81 kochR
0 54 moveto
3 81 kochR
0 81 moveto
1 81 kochR
stroke

```

Figure 5.13
Recursive PostScript for Koch
fractal

This modification to the PostScript program of Figure 4.3 transforms the output into a fractal (see text).

○ 5.25 Write a recursive program that fills in an n -by- 2^n array with 0s and 1s such that the array represents all the n -bit binary numbers, as depicted in Figure 5.9.

5.26 Draw the results of using the recursive ruler-drawing program (Program 5.8) for these unintended values of the arguments: `rule(0, 11, 4)`, `rule(4, 20, 4)`, and `rule(7, 30, 5)`.

5.27 Prove the following fact about the ruler-drawing program (Program 5.8): If the difference between its first two arguments is a power of 2, then both of its recursive calls have this property also.

○ 5.28 Write a function that computes efficiently the number of trailing 0s in the binary representation of an integer.

○ 5.29 How many squares are there in Figure 5.12 (counting the ones that are covered up by bigger squares)?

○ 5.30 Write a recursive C program that outputs a PostScript program that draws the bottom diagram in Figure 5.12, in the form of a list of function calls `x y r box`, which draws an r -by- r square at (x, y) . Implement `box` in PostScript (see Section 4.3).

5.31 Write a bottom-up nonrecursive program (similar to Program 5.9) that draws the bottom diagram in Figure 5.12, in the manner described in Exercise 5.30.

● 5.32 Write a PostScript program that draws the bottom diagram in Figure 5.12.

▷ 5.33 How many line segments are there in a Koch star of order n ?

● 5.34 Drawing a Koch star of order n amounts to executing a sequence of commands of the form “rotate α degrees, then draw a line segment of length $1/3^n$.” Find a correspondence with number systems that gives you a way to draw the star by incrementing a counter, then computing the angle α from the counter value.

● 5.35 Modify the Koch star program in Figure 5.13 to produce a different fractal based on a five-line figure for order 0, defined by 1-unit moves east, north, east, south, and east, in that order (see Figure 4.3).

5.36 Write a recursive divide-and-conquer function to draw an approximation to a line segment in an integer coordinate space, given the endpoints. Assume that all coordinates are between 0 and M . *Hint:* First plot a point close to the middle.

5.3 Dynamic Programming

An essential characteristic of the divide-and-conquer algorithms that we considered in Section 5.2 is that they partition the problem into independent subproblems. When the subproblems are not independent,

the situation is more complicated, primarily because direct recursive implementations of even the simplest algorithms of this type can require unthinkable amounts of time. In this section, we consider a systematic technique for avoiding this pitfall for an important class of problems.

For example, Program 5.10 is a direct recursive implementation of the recurrence that defines the Fibonacci numbers (see Section 2.3). *Do not use this program:* It is spectacularly inefficient. Indeed, the number of recursive calls to compute F_N is exactly F_{N+1} . But F_N is about ϕ^N , where $\phi \approx 1.618$ is the golden ratio. The awful truth is that Program 5.10 is an *exponential-time* algorithm for this trivial computation. Figure 5.14, which depicts the recursive calls for a small example, makes plain the amount of recomputation that is involved.

By contrast, it is easy to compute F_N in *linear* (proportional to N) time, by computing the first N Fibonacci numbers and storing them in an array:

```
F[0] = 0; F[1] = 1;
for (i = 2; i <= N; i++)
    F[i] = F[i-1] + F[i-2];
```

The numbers grow exponentially, so the array is small—for example, $F_{45} = 1836311903$ is the largest Fibonacci number that can be represented as a 32-bit integer, so an array of size 46 will do.

This technique gives us an immediate way to get numerical solutions for any recurrence relation. In the case of Fibonacci numbers, we can even dispense with the array, and keep track of just the previous two values (see Exercise 5.37); for many other commonly encountered recurrences (see, for example, Exercise 5.40), we need to maintain the array with all the known values.

A recurrence is a recursive function with integer values. Our discussion in the previous paragraph leads to the conclusion that we can evaluate any such function by computing all the function values in order starting at the smallest, using previously computed values at each step to compute the current value. We refer to this technique as *bottom-up dynamic programming*. It applies to any recursive computation, *provided* that we can afford to save all the previously computed values. It is an algorithm-design technique that has been used successfully for a wide range of problems. We have to pay attention to a

Program 5.10 Fibonacci numbers (recursive implementation)

This program, although compact and elegant, is not usable because it takes exponential time to compute F_N . The running time to compute F_{N+1} is $\phi \approx 1.6$ times as long as the running time to compute F_N . For example, since $\phi^9 > 60$, if we notice that our computer takes about a second to compute F_N , we know that it will take more than a minute to compute F_{N+9} and more than an hour to compute F_{N+18} .

```
int F(int i)
{
    if (i < 1) return 0;
    if (i == 1) return 1;
    return F(i-1) + F(i-2);
}
```

simple technique that can improve the running time of an algorithm from exponential to linear!

Top-down dynamic programming is an even simpler view of the technique that allows us to execute recursive functions at the same cost as (or less cost than) bottom-up dynamic programming, in an automatic way. We instrument the recursive program to save each value that it computes (as its final action), and to check the saved values to avoid recomputing any of them (as its first action). Program 5.11 is the mechanical transformation of Program 5.10 that reduces its running time to be linear via top-down dynamic programming. Figure 5.15 shows the drastic reduction in the number of recursive calls achieved by this simple automatic change. Top-down dynamic programming is also sometimes called *memoization*.

For a more complicated example, consider the *knapsack problem*: A thief robbing a safe finds it filled with N types of items of varying size and value, but has only a small knapsack of capacity M to use to carry the goods. The knapsack problem is to find the combination of items which the thief should choose for the knapsack in order to maximize the total value of all the stolen items. For example, with the item types depicted in Figure 5.16, a thief with a knapsack of size 17 can take five A's (but not six) for a total take of 20, or a D and an E for a total take of 24, or one of many other combinations. Our goal is

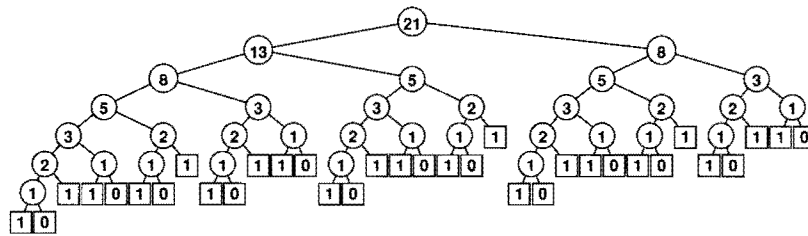


Figure 5.14
Structure of recursive algorithm for Fibonacci numbers

The picture of the recursive calls needed to compute F_8 by the standard recursive algorithm illustrates how recursion with overlapping subproblems can lead to exponential costs. In this case, the second recursive call ignores the computations done during the first, which results in massive recomputation because the effect multiplies recursively. The recursive calls to compute $F_6 = 8$ (which are reflected in the right subtree of the root and the left subtree of the left subtree of the root) are listed below.

```

8 F(6)
  5 F(5)
    3 F(4)
      2 F(3)
        1 F(2)
          1 F(1)
            0 F(0)
          1 F(1)
        1 F(2)
          1 F(1)
            0 F(0)
        2 F(3)
          1 F(2)
            1 F(1)
              0 F(0)
            1 F(1)
          3 F(4)
            2 F(3)
              1 F(2)
                1 F(1)
                  0 F(0)
                1 F(1)
              1 F(2)
                1 F(1)
                  0 F(0)

```

to find an efficient algorithm that somehow finds the maximum among all the possibilities, given any set of items and knapsack capacity.

There are many applications in which solutions to the knapsack problem are important. For example, a shipping company might wish to know the best way to load a truck or cargo plane with items for shipment. In such applications, other variants to the problem might arise as well: for example, there might be a limited number of each kind of item available, or there might be two trucks. Many such variants can be handled with the same approach that we are about to examine for solving the basic problem just stated; others turn out to be much more difficult. There is a fine line between feasible and infeasible problems of this type, which we shall examine in Part 8.

In a recursive solution to the knapsack problem, each time that we choose an item, we assume that we can (recursively) find an optimal way to pack the rest of the knapsack. For a knapsack of size cap , we determine, for each item i among the available item types, what total value we could carry by placing i in the knapsack with an optimal packing of other items around it. That optimal packing is simply the one we have discovered (or will discover) for the smaller knapsack of size $cap - \text{items}[i].\text{size}$. This solution exploits the principle that optimal decisions, once made, do not need to be changed. Once we know how to pack knapsacks of smaller capacities with optimal sets of items, we do not need to reexamine those problems, regardless of what the next items are.

Program 5.12 is a direct recursive solution based on this discussion. Again, this program is not feasible for use in solving actual problems, because it takes exponential time due to massive recomputation (see Figure 5.17), but we can automatically apply top-down dynamic programming to eliminate this problem, as shown in Pro-

Program 5.11 Fibonacci numbers (dynamic programming)

By saving the values that we compute in an array external to the recursive procedure, we explicitly avoid any recomputation. This program computes F_N in time proportional to N , in stark contrast to the $O(\phi^N)$ time used by Program 5.10.

```
int F(int i)
{ int t;
  if (knownF[i] != unknown) return knownF[i];
  if (i == 0) t = 0;
  if (i == 1) t = 1;
  if (i > 1) t = F(i-1) + F(i-2);
  return knownF[i] = t;
}
```

gram 5.13. As before, this technique eliminates all recomputation, as shown in Figure 5.18.

By design, dynamic programming eliminates all recomputation in *any* recursive program, subject only to the condition that we can afford to save the values of the function for arguments smaller than the call in question.

Property 5.3 *Dynamic programming reduces the running time of a recursive function to be at most the time required to evaluate the function for all arguments less than or equal to the given argument, treating the cost of a recursive call as constant.*

See Exercise 5.50. ■

For the knapsack problem, this property implies that the running time is proportional to NM . Thus, we can solve the knapsack problem easily when the capacity is not huge; for huge capacities, the time and space requirements may be prohibitively large.

Bottom-up dynamic programming applies to the knapsack problem, as well. Indeed, we can use the bottom-up approach any time that we use the top-down approach, although we need to take care to ensure that we compute the function values in an appropriate order, so that each value that we need has been computed when we need it. For functions with single integer arguments such as the two that we have

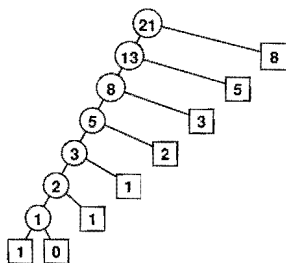


Figure 5.15
Top-down dynamic programming for computing Fibonacci numbers

This picture of the recursive calls used to compute F_8 by the top-down dynamic programming implementation of the recursive algorithm illustrates how saving computed values cuts the cost from exponential (see Figure 5.14) to linear.

Program 5.12 Knapsack problem (recursive implementation)

As we warned about the recursive solution to the problem of computing the Fibonacci numbers, *do not use this program*, because it will take exponential time and therefore may not ever run to completion even for small problems. It does, however, represent a compact solution that we can improve easily (see Program 5.13). This code assumes that items are structures with a size and a value, defined with

```
typedef struct { int size; int val; } Item;
```

and that we have an array of N items of type `Item`. For each possible item, we calculate (recursively) the maximum value that we could achieve by including that item, then take the maximum of all those values.

```
int knap(int cap)
{ int i, space, max, t;
  for (i = 0, max = 0; i < N; i++)
    if ((space = cap-items[i].size) >= 0)
      if ((t = knap(space) + items[i].val) > max)
        max = t;
  return max;
}
```

considered, we simply proceed in increasing order of the argument (see Exercise 5.53); for more complicated recursive functions, determining a proper order can be a challenge.

For example, we do not need to restrict ourselves to recursive functions with single integer arguments. When we have a function with multiple integer arguments, we can save solutions to smaller subproblems in multidimensional arrays, one for each argument. Other situations involve no integer arguments at all, but rather use an abstract discrete problem formulation that allows us to decompose problems into smaller ones. We shall consider examples of such problems in Parts 5 through 8.

In top-down dynamic programming, we save known values; in bottom-up dynamic programming, we precompute them. We generally prefer top-down to bottom-up dynamic programming, because

- It is a mechanical transformation of a natural problem solution.
- The order of computing the subproblems takes care of itself.
- We may not need to compute answers to all the subproblems.

	0	1	2	3	4
item	A	B	C	D	E
size	3	4	7	8	9
val	4	5	10	11	13

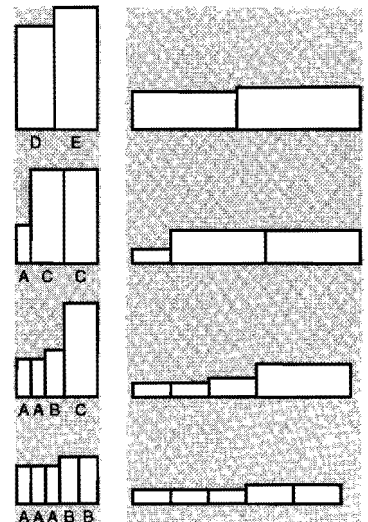
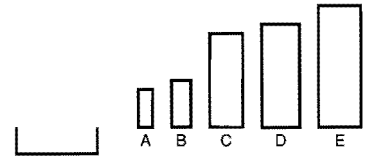


Figure 5.16
Knapsack example

An instance of the knapsack problem (top) consists of a knapsack capacity and a set of items of varying size (horizontal dimension) and value (vertical dimension). This figure shows four different ways to fill a knapsack of size 17, two of which lead to the highest possible total value of 24.

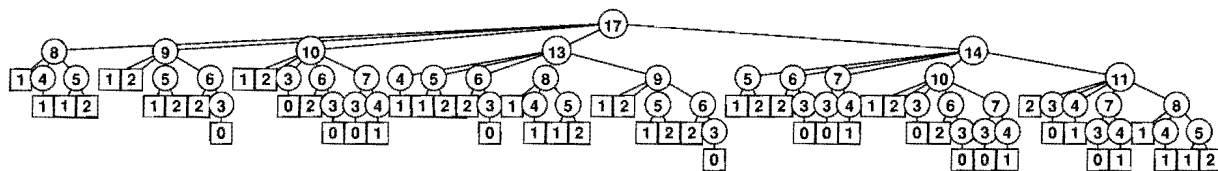


Figure 5.17
Recursive structure of knapsack algorithm.

This tree represents the recursive call structure of the simple recursive knapsack algorithm in Program 5.12. The number in each node represents the remaining capacity in the knapsack. The algorithm suffers the same basic problem of exponential performance due to massive recomputation for overlapping subproblems that we considered in computing Fibonacci numbers (see Figure 5.14).

Dynamic-programming applications differ in the nature of the subproblems and in the amount of information that we need to save regarding the subproblems.

A crucial point that we cannot overlook is that dynamic programming becomes ineffective when the number of possible function values that we might need is so high that we cannot afford to save (top-down) or precompute (bottom-up) all of them. For example, if M and the item sizes are 64-bit quantities or floating-point numbers in the knapsack problem, we will not be able to save values by indexing into an array. This distinction causes more than a minor annoyance—it poses a fundamental difficulty. No good solution is known for such problems; we will see in Part 8 that there is good reason to believe that no good solution exists.

Dynamic programming is an algorithm-design technique that is primarily suited for the advanced problems of the type that we shall consider in Parts 5 through 8. Most of the algorithms that we discuss in Parts 2 through 4 are divide-and-conquer methods with nonoverlapping subproblems, and we are focusing on subquadratic or sublinear, rather than subexponential, performance. However, top-down dynamic programming is a basic technique for developing efficient implementations of recursive algorithms that belongs in the toolbox of anyone engaged in algorithm design and implementation.

Exercises

- ▷ 5.37 Write a function that computes $F_N \bmod M$, using only a constant amount of space for intermediate calculations.
- 5.38 What is the largest N for which F_N can be represented as a 64-bit integer?
- 5.39 Draw the tree corresponding to Figure 5.15 for the case where we exchange the recursive calls in Program 5.11.
- 5.40 Write a function that uses bottom-up dynamic programming to compute the value of P_N defined by the recurrence

$$P_N = \lfloor N/2 \rfloor + P_{\lfloor N/2 \rfloor} + P_{\lceil N/2 \rceil}, \quad \text{for } N \geq 1 \text{ with } P_0 = 0.$$

Program 5.13 Knapsack problem (dynamic programming)

This mechanical modification to the code of Program 5.12 reduces the running time from exponential to linear. We simply save any function values that we compute, then retrieve any saved values whenever we need them (using a sentinel value to represent unknown values), rather than making recursive calls. We save the index of the item, so that we can reconstruct the contents of the knapsack after the computation, if we wish: `itemKnown[M]` is in the knapsack, the remaining contents are the same as for the optimal knapsack of size `M-itemKnown[M].size` so `itemKnown[M-items[M].size]` is in the knapsack, and so forth.

```
int knap(int M)
{ int i, space, max, maxi, t;
  if (maxKnown[M] != unknown) return maxKnown[M];
  for (i = 0, max = 0; i < N; i++)
    if ((space = M-items[i].size) >= 0)
      if ((t = knap(space) + items[i].val) > max)
        { max = t; maxi = i; }
  maxKnown[M] = max; itemKnown[M] = items[maxi];
  return max;
}
```

Draw a plot of N versus $P_N - N \lg N/2$ for $0 \leq N \leq 1024$.

5.41 Write a function that uses top-down dynamic programming to solve Exercise 5.40.

○ 5.42 Draw the tree corresponding to Figure 5.15 for your function from Exercise 5.41, when invoked for $N = 23$.

5.43 Draw a plot of N versus the number of recursive calls that your function from Exercise 5.41 makes to compute P_N , for $0 \leq N \leq 1024$. (For the purposes of this calculation, start your program from scratch for each N .)

5.44 Write a function that uses bottom-up dynamic programming to compute the value of C_N defined by the recurrence

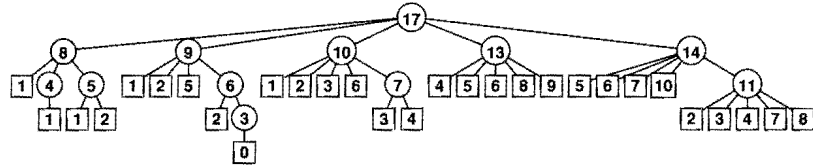
$$C_N = N + \frac{1}{N} \sum_{1 \leq k \leq N} (C_{k-1} + C_{N-k}), \quad \text{for } N \geq 1 \text{ with } C_0 = 1.$$

5.45 Write a function that uses top-down dynamic programming to solve Exercise 5.44.

○ 5.46 Draw the tree corresponding to Figure 5.15 for your function from Exercise 5.45, when invoked for $N = 23$.

Figure 5.18
Top-down dynamic programming for knapsack algorithm

As it did for the Fibonacci numbers computation, the technique of saving known values reduces the cost of the knapsack algorithm from exponential (see Figure 5.17) to linear.



- 5.47 Draw a plot of N versus the number of recursive calls that your function from Exercise 5.45 makes to compute C_N , for $0 \leq N \leq 1024$. (For the purposes of this calculation, start your program from scratch for each N .)
- ▷ 5.48 Give the contents of the arrays `maxKnown` and `itemKnown` that are computed by Program 5.13 for the call `knap(17)` with the items in Figure 5.16.
- ▷ 5.49 Give the tree corresponding to Figure 5.18 under the assumption that the items are considered in decreasing order of their size.
- 5.50 Prove Property 5.3.
- 5.51 Write a function that solves the knapsack problem using a bottom-up dynamic programming version of Program 5.12.
- 5.52 Write a function that solves the knapsack problem using top-down dynamic programming, but using a recursive solution based on computing the optimal number of a particular item to include in the knapsack, based on (recursively) knowing the optimal way to pack the knapsack without that item.
- 5.53 Write a function that solves the knapsack problem using a bottom-up dynamic programming version of the recursive solution described in Exercise 5.52.
- 5.54 Use dynamic programming to solve Exercise 5.4. Keep track of the total number of function calls that you save.
- 5.55 Write a program that uses top-down dynamic programming to compute the binomial coefficient $\binom{N}{k}$, based on the recursive rule

$$\binom{N}{k} = \binom{N-1}{k} + \binom{N-1}{k-1}$$

with $\binom{N}{0} = \binom{N}{N} = 1$.

5.4 Trees

Trees are a mathematical abstraction that play a central role in the design and analysis of algorithms because

- We use trees to describe dynamic properties of algorithms.

- We build and use explicit data structures that are concrete realizations of trees.

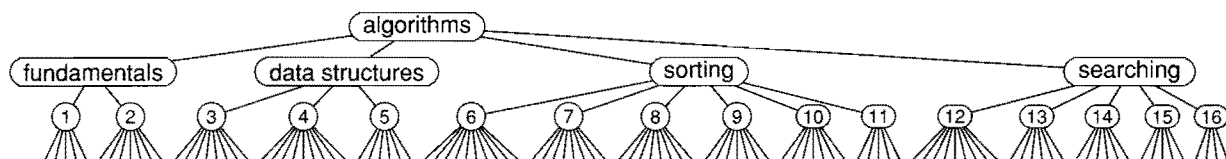
We have already seen examples of both of these uses. We designed algorithms for the connectivity problem that are based on tree structures in Chapter 1, and we described the call structure of recursive algorithms with tree structures in Sections 5.2 and 5.3.

We encounter trees frequently in everyday life—the basic concept is a familiar one. For example, many people keep track of ancestors or descendants with a family tree; as we shall see, much of our terminology is derived from this usage. Another example is found in the organization of sports tournaments; this usage was studied by Lewis Carroll, among others. A third example is found in the organizational chart of a large corporation; this usage is suggestive of the hierarchical decomposition that characterizes divide-and-conquer algorithms. A fourth example is a parse tree of an English sentence into its constituent parts; such trees are intimately related to the processing of computer languages, as discussed in Part 5. Figure 5.19 gives a typical example of a tree—one that describes the structure of this book. We touch on numerous other examples of applications of trees throughout the book.

In computer applications, one of the most familiar uses of tree structures is to organize file systems. We keep files in *directories* (which are also sometimes called *folders*) that are defined recursively as sequences of directories and files. This recursive definition again reflects a natural recursive decomposition, and is identical to the definition of a certain type of tree.

There are many different types of trees, and it is important to understand the distinction between the abstraction and the concrete representation with which we are working for a given application. Accordingly, we shall consider the different types of trees and their representations in detail. We begin our discussion by defining trees as abstract objects, and by introducing most of the basic associated terminology. We shall discuss informally the different types of trees that we need to consider in decreasing order of generality:

- Trees
- Rooted trees
- Ordered trees
- M -ary trees and binary trees

**Figure 5.19****A tree**

This tree depicts the parts, chapters, and sections in this book. There is a node for each entity. Each node is connected to its constituent parts by links down to them, and is connected to the large part to which it belongs by a link up to that part.

After developing a context with this informal discussion, we move to formal definitions and consider representations and applications. Figure 5.20 illustrates many of the basic concepts that we discuss and then define.

A *tree* is a nonempty collection of vertices and edges that satisfies certain requirements. A *vertex* is a simple object (also referred to as a *node*) that can have a name and can carry other associated information; an *edge* is a connection between two vertices. A *path* in a tree is a list of distinct vertices in which successive vertices are connected by edges in the tree. The defining property of a tree is that there is precisely one path connecting any two nodes. If there is more than one path between some pair of nodes, or if there is no path between some pair of nodes, then we have a graph; we do not have a tree. A disjoint set of trees is called a *forest*.

A *rooted tree* is one where we designate one node as the *root* of a tree. In computer science, we normally reserve the term *tree* to refer to rooted trees, and use the term *free tree* to refer to the more general structure described in the previous paragraph. In a rooted tree, any node is the root of a *subtree* consisting of it and the nodes below it.

There is exactly one path between the root and each of the other nodes in the tree. The definition implies no direction on the edges; we normally think of the edges as all pointing away from the root or all pointing towards the root, depending upon the application. We usually draw rooted trees with the root at the top (even though this convention seems unnatural at first), and we speak of node *y* as being *below* node *x* (and *x* as *above* *y*) if *x* is on the path from *y* to the root (that is, if *y* is below *x* as drawn on the page and is connected to *x* by a path that does not pass through the root). Each node (except the root) has exactly one node above it, which is called its *parent*; the nodes directly below a node are called its *children*. We sometimes carry the analogy to family trees further and refer to the *grandparent* or the *sibling* of a node.

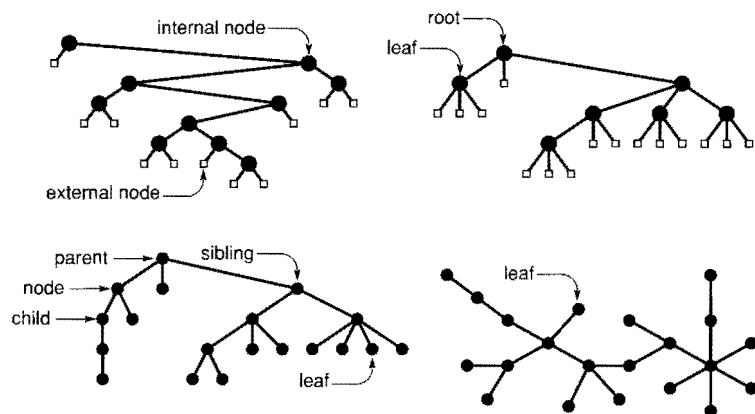


Figure 5.20
Types of trees

These diagrams show examples of a binary tree (top left), a ternary tree (top right), a rooted tree (bottom left), and a free tree (bottom right).

Nodes with no children are called *leaves*, or *terminal* nodes. To correspond to the latter usage, nodes with at least one child are sometimes called *nonterminal* nodes. We have seen an example in this chapter of the utility of distinguishing these types of nodes. In trees that we use to present the call structure of recursive algorithms (see, for example, Figure 5.14) the nonterminal nodes (circles) represent function invocations with recursive calls and the terminal nodes (squares) represent function invocations with no recursive calls.

In certain applications, the way in which the children of each node are ordered is significant; in other applications, it is not. An *ordered* tree is a rooted tree in which the order of the children at every node is specified. Ordered trees are a natural representation: for example, we place the children in some order when we draw a tree. As we shall see, this distinction is also significant when we consider representing trees in a computer.

If each node *must* have a specific number of children appearing in a specific order, then we have an *M-ary tree*. In such a tree, it is often appropriate to define special external nodes that have no children. Then, external nodes can act as dummy nodes for reference by nodes that do not have the specified number of children. In particular, the simplest type of *M-ary tree* is the binary tree. A *binary tree* is an ordered tree consisting of two types of nodes: external nodes with no children and internal nodes with exactly two children. Since the two children of each internal node are ordered, we refer to the *left child*

and the *right child* of internal nodes: every internal node must have both a left and a right child, although one or both of them might be an external node. A *leaf* in an M -ary tree is an internal node whose children are all external.

That is the basic terminology. Next, we shall consider formal definitions, representations, and applications of, in increasing order of generality,

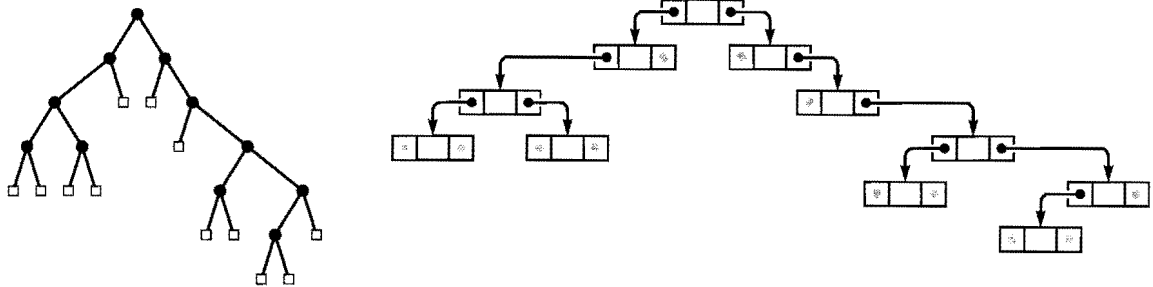
- Binary trees and M -ary trees
- Ordered trees
- Rooted trees
- Free trees

That is, a binary tree is a special type of ordered tree, an ordered tree is a special type of rooted tree, and a rooted tree is a special type of free tree. The different types of trees arise naturally in various applications, and is important to be aware of the distinctions when we consider ways of representing trees with concrete data structures. By starting with the most specific abstract structure, we shall be able to consider concrete representations in detail, as will become clear.

Definition 5.1 *A binary tree is either an external node or an internal node connected to a pair of binary trees, which are called the left subtree and the right subtree of that node.*

This definition makes it plain that the binary tree itself is an abstract mathematical concept. When we are working with a computer representation, we are working with just one concrete realization of that abstraction. The situation is no different from representing real numbers with floats, integers with ints, and so forth. When we draw a tree with a node at the root connected by edges to the left subtree on the left and the right subtree on the right, we are choosing a convenient concrete representation. There are many different ways to represent binary trees (see, for example, Exercise 5.62) that are surprising at first, but, upon reflection, that are to be expected, given the abstract nature of the definition.

The concrete representation that we use most often when we implement programs that use and manipulate binary trees is a structure with two links (a left link and a right link) for internal nodes (see Figure 5.21). These structures are similar to linked lists, but they have two links per node, rather than one. Null links correspond to



external nodes. Specifically, we add a link to our standard linked list representation from Section 3.3, as follows:

```
typedef struct node *link;
struct node { Item item; link l, r; };
```

which is nothing more than C code for Definition 5.1. Links are references to nodes, and a node consists of an item and a pair of links. Thus, for example, we implement the abstract operation *move to the left subtree* with a pointer reference such as $x = x \rightarrow l$.

This standard representation allows for efficient implementation of operations that call for moving *down* the tree from the root, but not for operations that call for moving *up* the tree from a child to its parent. For algorithms that require such operations, we might add a third link to each node, pointing to the parent. This alternative is analogous to a doubly linked list. As with linked lists (see Figure 3.6), we keep tree nodes in an array and use indices instead of pointers as links in certain situations. We examine a specific instance of such an implementation in Section 12.7. We use other binary-tree representations for certain specific algorithms, most notably in Chapter 9.

Because of all the different possible representations, we might develop a binary-tree ADT that encapsulates the important operations that we want to perform, and that separates the use and implementation of these operations. We do not take this approach in this book because

- We most often use the two-link representation.
- We use trees to implement higher-level ADTs, and wish to focus on those.
- We work with algorithms whose efficiency depends on a particular representation—a fact that might be lost in an ADT.

Figure 5.21
Binary-tree representation

The standard representation of a binary tree uses nodes with two links: a left link to the left subtree and a right link to the right subtree. Null links correspond to external nodes.

These are the same reasons that we use familiar concrete representations for arrays and linked lists. The binary-tree representation depicted in Figure 5.21 is a fundamental tool that we are now adding to this short list.

For linked lists, we began by considering elementary operations for inserting and deleting nodes (see Figures 3.3 and 3.4). For the standard representation of binary trees, such operations are not necessarily elementary, because of the second link. If we want to delete a node from a binary tree, we have to reconcile the basic problem that we may have two children to handle after the node is gone, but only one parent. There are three natural operations that do not have this difficulty: insert a new node at the bottom (replace a null link with a link to a new node), delete a leaf (replace the link to it by a null link), and combine two trees by creating a new root with a left link pointing to one tree and the right link pointing to the other one. We use these operations extensively when manipulating binary trees.

Definition 5.2 *An M -ary tree is either an external node or an internal node connected to an ordered sequence of M trees that are also M -ary trees.*

We normally represent nodes in M -ary trees either as structures with M named links (as in binary trees) or as arrays of M links. For example, in Chapter 15, we consider 3-ary (or *ternary*) trees where we use structures with three named links (left, middle, and right) each of which has specific meaning for associated algorithms. Otherwise, the use of arrays to hold the links is appropriate because the value of M is fixed, although, as we shall see, we have to pay particular attention to excessive use of space when using such a representation.

Definition 5.3 *A tree (also called an ordered tree) is a node (called the root) connected to a sequence of disjoint trees. Such a sequence is called a forest.*

The distinction between ordered trees and M -ary trees is that nodes in ordered trees can have any number of children, whereas nodes in M -ary trees must have precisely M children. We sometimes use the term *general tree* in contexts where we want to distinguish ordered trees from M -ary trees.

Because each node in an ordered tree can have any number of links, it is natural to consider using a linked list, rather than an array,